OWASP
Smart Contract Security

scs.owasp.org ↗

Security Handbook :

Web3 Attack Vectors Mapping

10

Security Handbook Series

GRANT SUPPORT • 2025-2026

ecosystem support program

Supported by Ethereum Foundation's Ecosystem Support Program (ESP)

Contents

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

Web3 Attack Vectors Mapping

About This Handbook

Web3 Attack Vectors Mapping is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook builds a structured map of Web3 attack vectors that runs from the wallet and the user sitting in front of it, through the dApp front end, into the smart contract layer, and out to the node and infrastructure that keep a chain running. It is the companion volume to the OWASP Web3 Attack Vectors Top 15, the Alternate Top 15 list published alongside the OWASP Smart Contract Top 10 (SC Top 10) at scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15. Where the SC Top 10 catalogs on-chain logic risk, the Top 15 (WA01 through WA15) is an awareness list of the significant vectors that sit outside contract code: custody failures, social engineering, drainer malware, DNS hijacking, insider abuse, and nation-state operations against the people and pipelines that build Web3 software.

It organizes those vectors into a tactic-technique-procedure (TTP) catalog modeled on MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) and extended by MITRE AADAPT, MITRE's companion framework for digital asset and blockchain payment technologies. Each layer of the stack (user, wallet, dApp, contract, node, infrastructure) gets its own technique catalog, cross-referenced to the Top 15 vector IDs, to the Smart Contract Security Verification Standard (SCSVS), and to the Smart Contract Weakness Enumeration (SCWE), so a threat-modeling exercise or a detection team can work end to end from a single reference instead of stitching together five separate documents.

Target Audience

Security specialists, threat intelligence teams, and architects performing threat modeling across the Web3 stack.

How to Use This Handbook

Start with Part I to fix the taxonomy: the six layers this handbook uses, how they map onto WA01 through WA15, and the STRIDE-based threat-modeling method the rest of the book assumes. Part II is the technique catalog itself, one chapter per layer, and each chapter opens with its Top 15 alignment before working through specific techniques and their TTP IDs. Part III turns that catalog into a usable matrix: how to read it, how to apply it in a threat-modeling session, and how to apply it during detection and incident response. Part IV grounds the framework in named incidents, among them Bybit, the Ledger Connect Kit compromise, drainer-as-a-service campaigns, the Trust Wallet extension incident, and DPRK's TraderTraitor operation, traced across multiple layers at once. A reader building a detection program can go straight to the Part III

matrix and the Part V quick-reference appendices; a reader running a single threat-modeling workshop should read Part I first, then the layer chapters that match the system's actual attack surface.

Relationship to SCSVS, SCSTG, and SCWE

This handbook sits above the three core Smart Contract Security (SCS) standards as the connective layer between them. SCSVS defines the controls a system must satisfy and the Smart Contract Security Testing Guide (SCSTG) defines how to test for them, but both are scoped to the contract and its immediate surroundings; the TTP catalog here extends threat coverage to the wallets, front ends, RPC endpoints, and infrastructure that SCSVS and SCSTG do not reach. SCWE catalogs on-chain weaknesses, and Chapter 7 maps this handbook's contract-layer techniques directly onto SCWE entries instead of duplicating them, while the other five layers fill the ground the enumeration leaves uncovered. Two sibling handbooks depend directly on this mapping: the CDN and Front-End Supply Chain Security Handbook (01) supplies the deep technical detail behind the dApp-layer techniques in Chapter 6, and the Incident Response Handbook (06) consumes the detection and mitigation columns of the Part III matrix as its triage reference.

Part I: Foundations

This part sets the vocabulary the rest of the handbook depends on: the six layers a Web3 system spans, the tactics-techniques-procedures (TTP) model this handbook borrows from MITRE ATT&CK and MITRE AADAPT, and the OWASP Web3 Attack Vectors Top 15 that anchors every chapter downstream. It treats an attack as a chain that crosses layers rather than a single-point event, then gives you the threat-modeling method (STRIDE plus scenario mapping) that Part 3 and Part 4 apply repeatedly. Read this part once before using the rest of the handbook as reference.

1. Introduction to Web3 Attack Vector Mapping

A Web3 system is not one artifact to defend. It is a chain of six: a person, the keys that person holds, the interface they click through, the contract that interface calls, the node that relays the call, and the infrastructure that keeps all of it online. This chapter introduces the frameworks this handbook borrows to describe attacks against that chain and fixes how it relates to OWASP's existing smart contract standards.

1.1 Why a Unified View (Wallets to Infra)

Security teams in Web3 default to organizing around one layer, usually the smart contract, because that is where audits and bug bounties concentrate. Real losses rarely stay inside one layer. Consider the theft of roughly $1.4 billion in ether from Bybit on 21 February 2025, the largest cryptocurrency heist recorded to date. The attacker did not touch Bybit's contracts. Attackers compromised a developer's machine at Safe{Wallet}, the multisignature wallet interface Bybit's signers used, and injected JavaScript that showed signers a routine transfer while the actual payload was a `delegatecall` repointing the cold wallet's proxy implementation to attacker-controlled logic. Signers blind-signed calldata they could not read, approved what looked routine, and the funds moved. Investigators attributed the operation to North Korea's Lazarus Group based on the laundering pattern that followed (NCC Group technical analysis).

That single incident spans four layers: the user (a signer who trusted what the screen showed), the wallet (the multisignature approval flow), the dApp front end (the compromised interface serving malicious JavaScript), and the infrastructure behind it (the developer workstation that gave the attacker a foothold). A team that only threat-models the Safe contract itself, which behaved exactly as written, would never have found this path. A defender reasoning about the full chain asks the same question at every hop: what does this layer trust from the layer before it, and what happens if that trust breaks? This handbook makes that question repeatable, and later parts revisit Bybit and comparable incidents (Ledger Connect Kit, covered in the CDN and Front-End Supply Chain Security Handbook, 01) as worked examples of chains a single-layer view misses.

1.2 MITRE ATT&CK and TTP Concepts

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) is, in its own words, "a globally-accessible knowledge base of adversary tactics and techniques based on real-world observations" (attack.mitre.org). It organizes adversary behavior into a hierarchy that this handbook reuses throughout: a **tactic** is the adversary's short-term goal, a **technique** is a specific method used to achieve that goal, a **sub-technique** is a more granular variant of a technique, and a **procedure** is the exact implementation a particular threat actor used in a particular incident. NIST SP 800-150 formalizes the umbrella term TTP (tactics, techniques, and procedures) this way: a tactic is "the highest-level description" of adversary behavior, techniques "give a more detailed description… in the context of a tactic," and procedures are "an even lower-level, highly detailed description in the context of a technique."

The ATT&CK Enterprise matrix currently defines 14 tactics, from Reconnaissance and Initial Access through Impact, each identified by a stable `TAnnnn` ID, with techniques and sub-techniques identified by `Tnnnn` and `Tnnnn.nnn` respectively.

Level	Granularity	Example ID	Example
Tactic	Adversary's goal	TA0001	Initial Access
Technique	Method to reach the goal	T1566	Phishing
Sub-technique	Specific variant of the technique	T1566.002	Spearphishing Link
Procedure	Exact implementation in one incident	(free text, no fixed ID)	A cloned wallet-connect page sent to a specific signer with a lookalike domain

This handbook adopts the same tactic-technique-procedure structure for every layer in Part 2, assigns each technique a handbook-local ID (Section 2.3), and cross-references ATT&CK and AADAPT IDs wherever an entry maps cleanly onto them.

1.3 MITRE AADAPT: Digital Asset and Blockchain Threat Framework

ATT&CK was built for enterprise IT, not for systems where the adversary's objective is the irreversible transfer of a bearer asset. MITRE's Center for Threat-Informed Defense addressed that gap with AADAPT, Adversarial Actions in Digital Asset Payment Technologies, a knowledge base hosted at aadapt.mitre.org and published as open data in the `mitre/AADAPT` GitHub repository. AADAPT is explicitly modeled on ATT&CK's structure (tactics, techniques, a navigable matrix) but scoped to cryptocurrency, blockchain, and digital-asset payment systems, the domain ATT&CK does not cover.

1.3.1 Complement to ATT&CK; Real-World Attack Derivation

AADAPT's published data (`AADAPT.yaml`, retrieved August 2026) shows how it complements rather than replaces ATT&CK. Ten of its eleven tactics are ATT&CK Enterprise tactics reused with their original `TAnnnn` IDs: Reconnaissance, Resource Development, Initial Access, Execution, Privilege Escalation, Defense Evasion, Credential Access, Lateral Movement, Collection, and Impact. AADAPT drops four ATT&CK tactics that fit general IT intrusions poorly here (Persistence, Discovery, Command and Control, Exfiltration) and adds one ATT&CK has no equivalent for: **Fraud** (`ADTA0001`), an adversary trying "to illicitly create, acquire, or utilize value-form," or to destroy a victim's value-form outright. Beneath these eleven tactics sit 66 techniques with `ADTnnnn` IDs, for example `ADT3001` (Acquire Accounts, under Resource Development) and `ADT3002` (Aggregate Private Key Generation Data, under Collection).

The "real-world attack derivation" in this subsection's title is not a slogan. Every AADAPT technique carries a references list pointing to named incidents rather than abstractions: the Ronin Network bridge exploit, the Euler Finance flash-loan attack, the FTX collapse, and the Bybit hack (the same NCC Group analysis cited in Section 1.1) all appear as sourced technique evidence. That is why this handbook cross-references AADAPT technique IDs in Part 2's wallet, node, and infrastructure chapters: an AADAPT ID points to a documented incident, not a hypothetical.

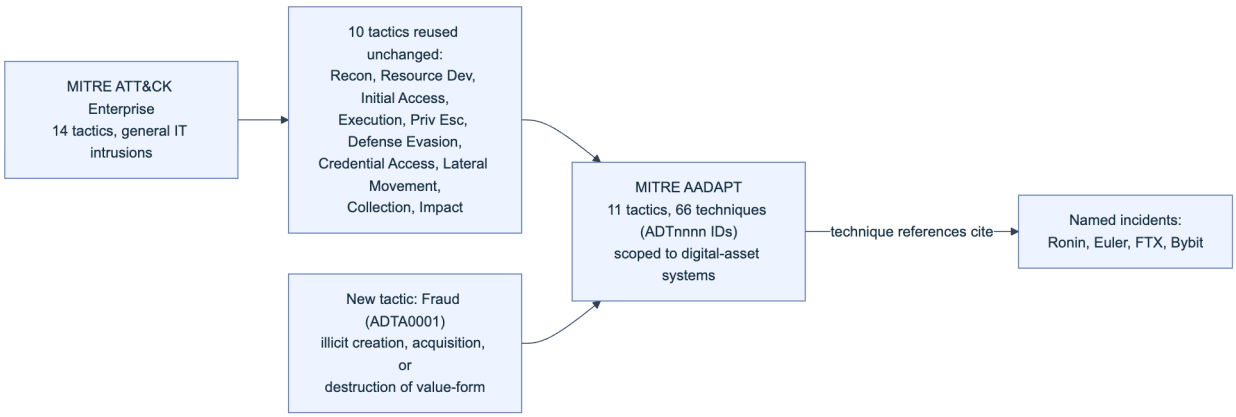

Figure 1. How AADAPT derives from ATT&CK: it keeps the ten tactics that transfer cleanly to digital-asset systems, adds one new tactic (Fraud) that ATT&CK has no category for, and grounds every technique in a cited incident.

1.4 Relationship to OWASP SC Top 10, SCWE, and SCSVS

Three existing OWASP Smart Contract Security (SCS) project deliverables already cover the contract layer in depth, and this handbook is built to extend that coverage outward, not duplicate it. The Smart Contract Top 10 ranks the highest-impact classes of on-chain logic risk (reentrancy, access control failures, oracle manipulation, flash-loan-facilitated attacks, and similar categories) for a given year. The Smart Contract Security Verification Standard (SCSVS) turns that risk landscape into a checklist of testable controls, organized into eleven domains from architecture through cryptography and component security. The Smart Contract Weakness Enumeration (SCWE) is the weakness-level catalog beneath both, naming specific patterns (a missing check, an unbounded loop, an unvalidated external call) and mapping each to the SCSVS control it violates.

All three are scoped, by design, to the contract and its immediate surroundings. This handbook's Chapter 7 (Smart Contract and Protocol, Part 2) maps directly onto SCWE's categories rather than re-deriving them, and the other five layers cover exactly the ground those three standards leave uncovered: the human who signs, the key that signs, the interface that requests the signature, the node that broadcasts it, and the infrastructure underneath. Where a technique here does touch contract logic (Section 2.4), it cites the relevant SCWE entry rather than restating it.

1.5 The OWASP Web3 Attack Vectors Top 15 (Alternate List)

The Web3 Attack Vectors Top 15 (WA01 through WA15) is the OWASP SCS project's companion awareness list to the Smart Contract Top 10, and it is the single most important external reference this handbook aligns to. Where the Smart Contract Top 10 is a ranked list of on-chain risk, the Top 15 is an unranked catalog of everything a Web3 organization loses money to that has nothing to do with a Solidity bug.

1.5.1 Canonical List: https://scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15/

The authoritative source is published at scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15 under the title "Alternate Top 15, Web3 Attack Vectors (Beyond Smart Contracts)." Treat every WAnn reference in this handbook as a pointer to that page rather than to a frozen snapshot: the OWASP SCS project states explicitly that the list will evolve (Section 1.5.5), so verify the current wording against the canonical page before citing a specific WA entry in a client-facing deliverable.

1.5.2 Scope: Non-Smart-Contract Vectors; Complements SC Top 10

The Top 15's own framing is unambiguous: it "complements the OWASP Smart Contract Top 10: 2026 by cataloguing significant non-smart-contract attack vectors in the Web3 space." That boundary is why this handbook exists as a separate volume rather than a chapter bolted onto the Smart Contract Top 10. A reentrancy bug is a code defect with a code fix. A drainer campaign, a fake-interview social-engineering operation, or a hijacked DNS record is not a code defect at all, it is a failure at the human, operational, or infrastructure layer, and it needs its own taxonomy, detection signals, and owner inside a security organization. The Top 15 draws that boundary explicitly so a reader does not mistake "our contracts passed audit" for "we are covered."

1.5.3 WA01–WA15 Overview and Summary Table

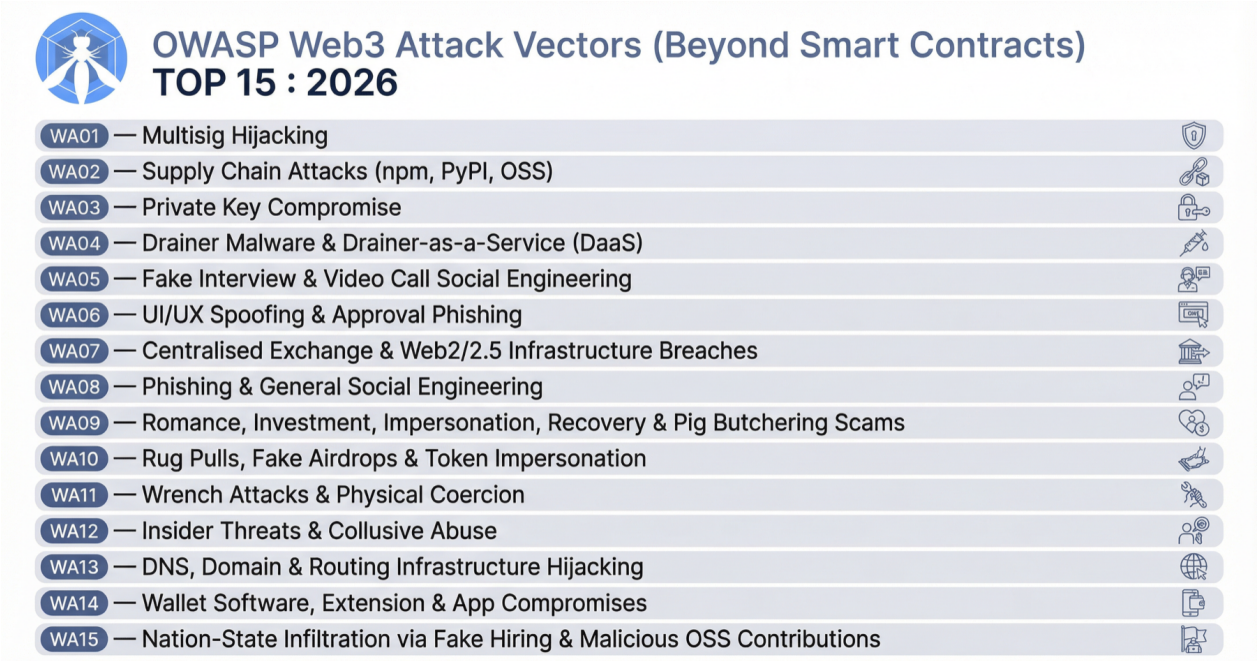

Figure. The canonical attack-surface view this handbook operationalizes. The list deliberately spans people, software, signing UX, custody, infrastructure, and state-backed infiltration, preventing a threat model from collapsing into smart-contract code alone. Source: OWASP SCS, Web3 Attack Vectors Top 15, OWASP project content.

ID	Vector	What it covers
WA01	Multisig Hijacking	Manipulated signing UIs trick operators into authorizing malicious transactions without seeing the real payload.
WA02	Supply Chain Attacks (npm, PyPI, OSS)	Compromised open-source packages distribute crypto-stealing malware to developers and users.
WA03	Private Key Compromise	Keys or seed phrases stolen via phishing, malware, or weak implementation; cited as roughly 43.8 percent of hacking-related losses.
WA04	Drainer Malware and Drainer-as-a-Service (DaaS)	Packaged phishing-site toolkits trick users into granting malicious contracts transfer permissions.
WA05	Fake Interview and Video Call Social Engineering	Attackers pose as employers, use fake meeting software, and abuse remote-control features to plant malware.
WA06	UI/UX Spoofing and Approval Phishing	Spoofed or cloned sites trick users into signing transactions granting spending authority to a malicious contract.
WA07	Centralised Exchange and Web2/2.5 Infrastructure Breaches	Custody and operational-security failures at exchanges and platforms enable large-scale theft.
WA08	Phishing and General Social Engineering	Phishing emails and fake support sites trick users into revealing seed phrases or passwords directly.
WA09	Romance, Investment, Impersonation, Recovery, and Pig Butchering Scams	Long-horizon social engineering builds trust to extract funds through fake platforms or impersonation.

ID	Vector	What it covers
WA10	Rug Pulls, Fake Airdrops, and Token Impersonation	Fraudulent launches and airdrop claims lure users into acquiring worthless or malicious tokens.
WA11	Wrench Attacks and Physical Coercion	Threats, kidnapping, or coercion force individuals to reveal credentials or authorize transfers.
WA12	Insider Threats and Collusive Abuse	Employees or contractors abuse legitimate wallet, admin-panel, or deployment access to steal funds.
WA13	DNS, Domain, and Routing Infrastructure Hijacking	Compromised DNS or registrars redirect users to phishing sites impersonating a real dApp or exchange.
WA14	Wallet Software, Extension, and App Compromises	Malicious updates or exploited vulnerabilities enable silent approval or key theft.
WA15	Nation-State Infiltration via Fake Hiring and Malicious OSS Contributions	State-backed groups pose as developers or employers to infiltrate orgs and compromise software supply chains.

Table 1. The OWASP Web3 Attack Vectors Top 15. Source: OWASP SCS, Alternate Top 15.

1.5.4 Use of the Top 15: Awareness, Threat Modeling, and Detection

The Top 15's authors position it as "an awareness list for protocols, Web3 users, stakeholders, and the CXO suite," and this handbook operationalizes that in three ways. As an **awareness checklist**, walk WA01 through WA15 during onboarding or vendor review and confirm each vector has a named owner, even if the answer for some is "accepted risk, out of scope." As a **threat-modeling input**, use the WA taxonomy to keep a STRIDE session (Section 3.2) from silently collapsing into a contract-only exercise; every WA entry maps to at least one layer in Section 2.2's table, so a session that never touches a WA vector has probably only covered the contract layer. As a **detection input**, each vector implies monitoring surface (seed-phrase entry on a foreign domain for WA08, an unexpected `setApprovalForAll` for WA06), so Part 3's TTP matrix maps WA vectors to concrete signals instead of leaving them as narrative categories.

- Assign an owner to each of WA01 to WA15 (security, legal, HR, or "accepted risk").
- Confirm at least one control exists per vector, even a manual process, before calling it covered.
- Re-run the mapping whenever the canonical page updates (Section 1.5.5).
- Feed WA IDs into the STRIDE and risk tables in Chapter 3 so nothing is threat-modeled twice under different names.

1.5.5 Future Development: Ranking, Methodology, and Update Cycles

The canonical page states plainly that the current ordering "does not imply relative severity or prevalence" and that the list "will be developed, ranked, and maintained in the near future," with methodology, data sources, and update cycles established as the project matures. Treat WA01 to WA15 as an index, not a priority order: do not tell a client that WA01 (Multisig Hijacking) outranks WA08 (Phishing) purely on its lower number. When OWASP publishes a ranked suc-

cessor, Section 2.2's layer mapping and the Part 3 TTP matrix are the two artifacts to re-validate; both are structured so adding, splitting, or re-ranking a WA entry does not require rebuilding the six-layer taxonomy underneath them.

1.6 How to Use This Handbook

Read this Part once, end to end, to internalize the six-layer taxonomy, the TTP vocabulary, and the STRIDE method. After that, use the handbook as a reference rather than a cover-to-cover read.

If your goal is…	Start here
Build or update a layer-by-layer technique catalog	Part 2, the chapter matching the layer in question
Run a threat-modeling workshop for a specific system	Part 1, Chapter 3, then the matching Part 2 chapters
Build a detection or incident-response playbook	Part 3's TTP matrix, then the Incident Response Handbook (06)
Understand a named incident across layers	Part 4's case studies
Look up a term, a WA ID, or an SCWE cross-reference fast	Part 5's appendices

Cross-references throughout point to sibling handbooks by number (02, 06, 07, 08, 09, 11) so that when a technique here touches a topic covered in depth elsewhere, you can follow the thread outward instead of finding a shallow restatement.

---

2. Scope and Taxonomy

This chapter fixes the six-layer model the handbook is built on and shows how it connects to the Top 15, to the handbook's own TTP identifiers, and to sibling handbooks.

2.1 Layers: User, Wallet, dApp, Contract, Node, Infra

Six layers cover the full path a transaction takes from human intent to settled state, each a distinct trust boundary with its own adversaries and controls, mirroring the level-based approach the CDN and Front-End Supply Chain Security Handbook (01) uses for its narrower scope.

The **user and social layer** is the human: trader, signer, developer, executive, and everything a social engineer can target in them. The **wallet and key management layer** is the private key or seed material and the software (browser extension, mobile app, hardware device, or multisignature contract) that signs with it. The **dApp and front-end layer** is the web or mobile interface that constructs transactions and requests signatures, the layer the Bybit incident (Section 1.1) compromised. The **smart contract and protocol layer** is the on-chain logic SCWE already catalogs in depth (Section 1.4). The **node and RPC (remote procedure call) layer** re-

ceives signed transactions, relays them to the network, and serves state back to clients. The **infrastructure layer** is everything underneath: cloud accounts, DNS, CI/CD (continuous integration and continuous deployment) pipelines, and, for centralized platforms, the exchange backend itself.

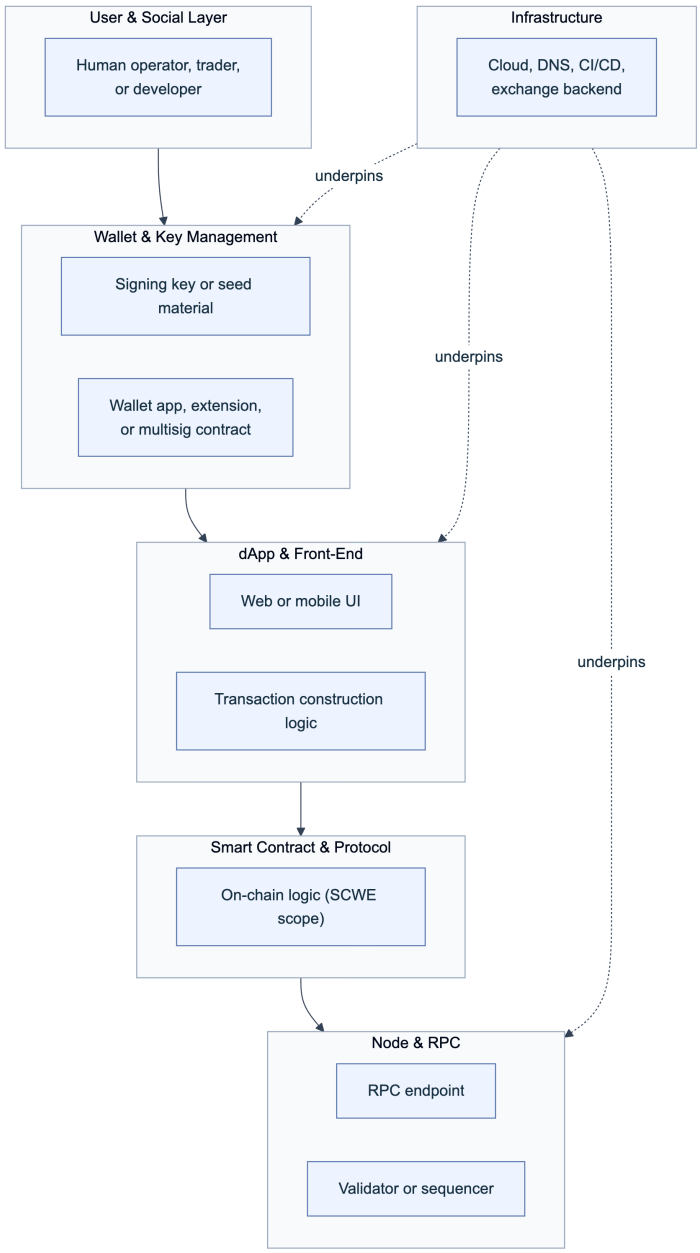

Figure 2. The six-layer taxonomy. Intent flows left to right from user through settlement; infrastructure underlies the wallet, dApp, and node layers rather than sitting only at one end of the chain.

2.2 Mapping Layers to the Web3 Attack Vectors Top 15 (WA01–WA15)

Most WA vectors have one primary layer and, often, a secondary layer where the attack lands. Use this table as the crosswalk between Table 1 and the layer chapters in Part 2.

Layer	Primary WA vectors	Secondary or cross-layer
User and social	WA05, WA08, WA09, WA11	WA04, WA06 (delivery mechanism)
Wallet and key management	WA01, WA03, WA04, WA06, WA14	WA02 (compromised wallet dependency)
dApp and front-end	WA02, WA06, WA10, WA13	WA01 (multisig UI), WA04 (drainer sites)
Smart contract and protocol	(none primary; see Section 1.4)	WA10 (fraudulent token contracts)
Node and RPC	(none primary in current Top 15)	WA13 (routing hijack can target RPC)
Infrastructure	WA07, WA12, WA13, WA15	WA02 (build infrastructure)

Table 2. Layer-to-vector crosswalk. A vector with no primary layer is fully covered elsewhere (contract logic by SCWE) or has no dedicated Top 15 entry yet (node-layer attacks are still covered in Part 2 Chapter 8).

2.3 Tactics and Techniques Overview

Part 2's six layer chapters each build a technique catalog using the TTP structure from Section 1.2, and every entry carries a handbook-local identifier so the Part 3 matrix, the Part 4 case studies, and the Part 5 appendices can reference it without ambiguity. The ID format is:

```
W3.<LAYER>.<NNN>

LAYER codes: US (User & Social) | WK (Wallet & Key Mgmt) | DA (dApp & Front-End)
             SC (Smart Contract) | ND (Node & RPC)        | IF (Infrastructure)
NNN: sequential three-digit number within the layer, assigned in Part 2

Example: W3.WK.004  ->  Wallet & Key Management layer, technique 004
```

Where a technique corresponds to an existing framework entry, the catalog cites it alongside the handbook ID rather than replacing it: an ATT&CK `Tnnnn` ID, an AADAPT `ADTnnnn` ID, a Top 15 `WAnn` ID, or an SCWE identifier for contract-layer entries. A technique with no clean external match still gets a `W3.*` ID and a plain-language justification. This keeps the catalog navigable by layer, by tactic (the Part 3 matrix's rows), and by external standard (the Part 5 appendix cross-reference tables).

2.4 Cross-References to Other Handbooks and SCWE

No layer here is this handbook's alone to own. Each has a sibling handbook that treats it in operational depth, and this handbook's job is to map the attack surface, not duplicate control detail those handbooks already carry.

Layer	Primary sibling handbook(s)	SCWE relevance
User and social	Employee Lifecycle Security (03); Hiring, Remote Work, and Insider Threat (05); UX Security (09)	None
Wallet and key management	UX Security (09); Web3 Operational Security (11)	None
dApp and front-end	CDN and Front-End Supply Chain Security (01); DNS and Hosting Security (02)	None
Smart contract and protocol	(mapping only; audits are out of scope)	Direct: all 11 SCWE categories
Node and RPC	Infrastructure Security (07)	Partial: SCSVS-BRIDGE, SCSVS-BLOCK for chain-facing logic
Infrastructure	DNS and Hosting Security (02); Infrastructure Security (07); Incident Response (06)	None

Table 3. Cross-reference map. When a Part 2 entry names a control this handbook does not detail, an SRI (Subresource Integrity) pin or a DNSSEC record, it points to the sibling handbook that owns it rather than restating it.

3. Threat Modeling and Attack Scenarios

This chapter supplies the method the rest of the handbook assumes: scoping a threat model across six layers, running it with STRIDE, visualizing a multi-layer attack, and turning the result into a prioritized mitigation list.

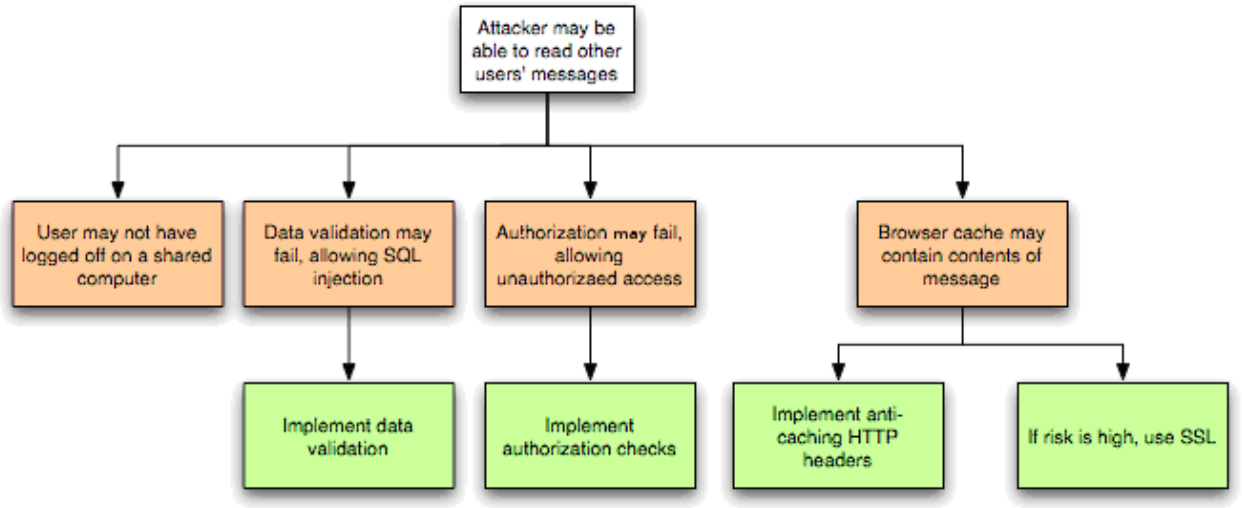

Figure. OWASP's threat graph makes the causal chain explicit: an agent uses a vector to exploit a weakness, producing technical and then business impact. In this handbook, the six-layer taxonomy supplies the locations, while the TTP matrix supplies the reusable vectors and controls. Source: OWASP Threat Modeling Process, OWASP community content used with attribution.

3.1 Define Scope, Assets, and Boundaries

Every threat model starts by writing down three things before a single attack is discussed: what is in scope, what an adversary would want, and where the trust boundaries sit. Scope is a decision, not a discovery: state explicitly whether the exercise covers one dApp, one organization's whole stack, or a single layer in isolation, because an unscoped session drifts toward whatever layer the loudest participant knows best. Assets in a Web3 system are rarely just "funds," enumerate them by layer: signing keys (wallet), session cookies and approval state (dApp), admin roles and upgrade keys (contract), RPC credentials (node), and DNS zone access and CI/CD secrets (infrastructure). Boundaries are the points where one layer's output becomes another layer's trusted input without independent verification: does the wallet show the human what it is actually signing? Does the contract validate what the front end sends, or trust it? Can an infrastructure compromise silently swap which node a client talks to?

Layer	Primary asset to protect	Trust boundary crossed
User and social	Attention and judgment	Human trusts a message, call, or site as legitimate
Wallet and key management	Private key or seed material	Wallet trusts the calldata it is asked to sign
dApp and front-end	Session state, approval scope	User trusts the interface to represent the transaction accurately
Smart contract and protocol	Contract state, admin roles	Contract trusts caller-supplied input and upstream oracles

Layer	Primary asset to protect	Trust boundary crossed
Node and RPC	Chain state, RPC responses	Client trusts the node it queries to be unmodified and honest
Infrastructure	DNS, cloud credentials, CI/CD secrets	Every layer above trusts infrastructure to be unmodified

Table 4. A scoping worksheet: fill in the asset and boundary columns for the system under review before running STRIDE.

3.2 STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege)

STRIDE is Microsoft's threat-categorization model, still the reference method for structured threat modeling nearly three decades after its introduction (Microsoft Learn, Threat Modeling Tool threat categories). Working through all six categories against every layer prevents the common failure of hardening the layer a team already worries about while an adjacent category goes unexamined; this handbook applies it across all six layers, not the single application boundary STRIDE was originally built for.

STRIDE category	Multi-layer Web3 scenario	Layer(s) affected	Related Top 15 vector
Spoofing	Cloned dApp domain or fake wallet-connect prompt impersonates the real interface	dApp, user	WA06, WA13
Tampering	Front-end JavaScript altered to change calldata before signing (as in Bybit)	dApp, wallet	WA01, WA14
Repudiation	An insider with deployment access pushes a malicious upgrade with no attributable trail	Infrastructure, contract	WA12
Information disclosure	CI/CD secrets or keys leak from a build pipeline or exposed RPC endpoint	Infrastructure, node	WA02, WA07
Denial of service	A targeted RPC provider or DNS host is knocked offline, stranding users	Node, infrastructure	WA13
Elevation of privilege	A phished support agent resets a user's two-factor authentication and drains a custodial balance	User, infrastructure	WA07, WA08

Table 5. STRIDE mapped across the six-layer taxonomy, with each scenario tied to a Top 15 vector and the layers it actually touches.

Run this table live in a workshop: replace each generic scenario with the team's own, and leave a cell blank only when the team can state affirmatively why that category does not apply at that layer, not because nobody thought about it.

3.3 Attack Scenarios and Visualizations

A STRIDE table tells you what can happen; a sequence diagram tells you in what order and where a control could have interrupted it. Build one for every scenario that crosses more than one layer, the class of attack a single-layer review misses. The example below walks a fake-airdrop lure (WA10) through to an on-chain drain, crossing the user, wallet, dApp, and contract layers.

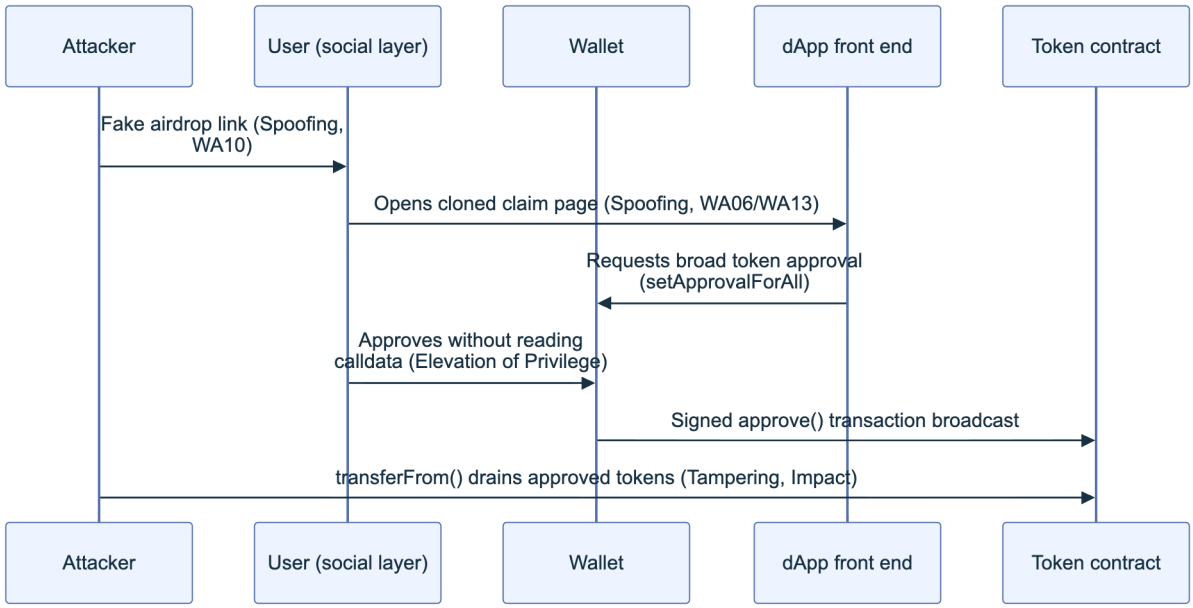

Figure 3. A worked multi-layer scenario. Each arrow is a point where a control (SRI on the claim page, a wallet that decodes calldata in plain language, an approval-scope limit) could have broken the chain; Part 3's matrix lists the concrete control for each step.

Attack trees and kill-chain diagrams (the Lockheed Martin model, adapted here to layer transitions instead of network stages) serve scenarios with branching paths rather than a single line; use whichever visualization makes the decision points visible to the people who act on the model.

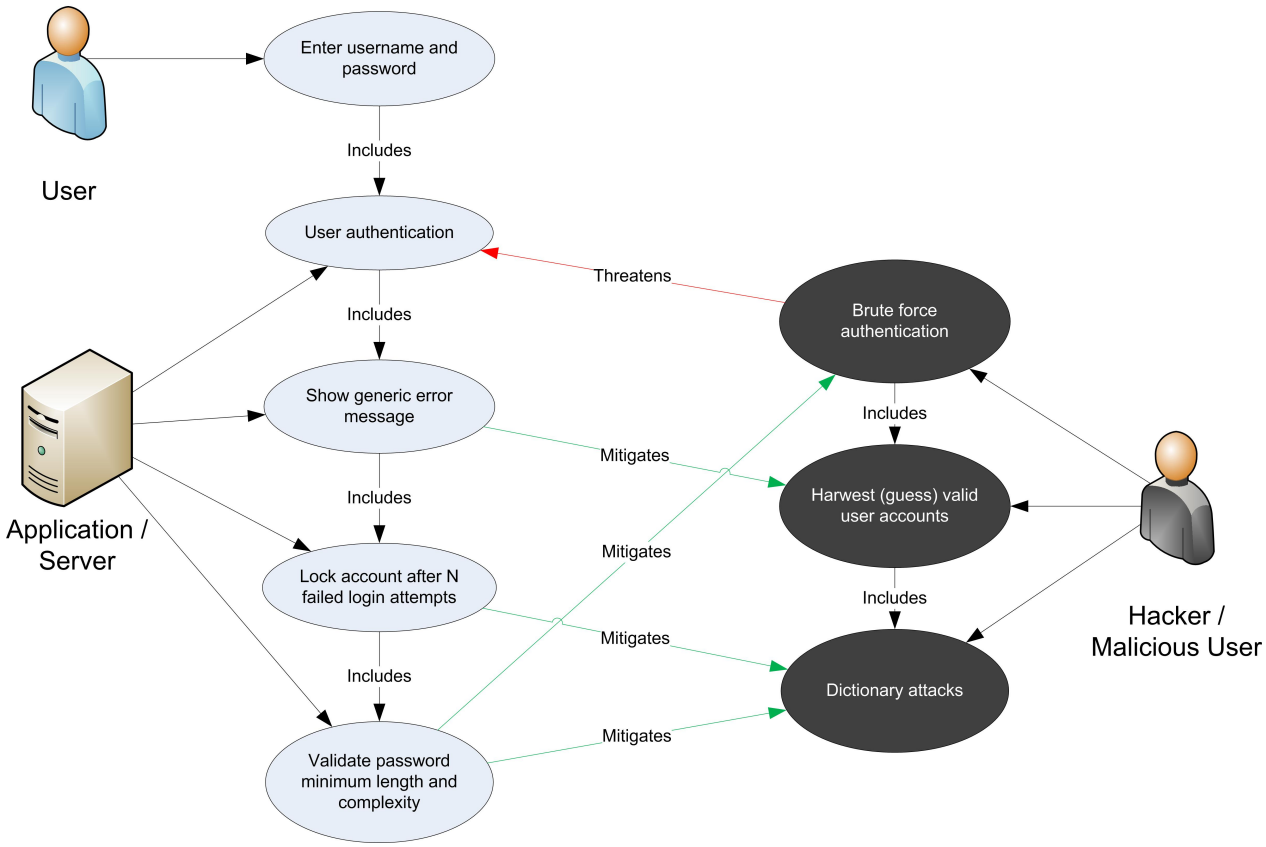

Figure. A misuse case anchors an adversary action to the normal function it abuses and the security behavior that must interrupt it. This is particularly useful for Web3 actions such as token approval, multisig signing, and contract upgrade, where the same valid mechanism serves both legitimate and malicious intent. Source: OWASP Threat Modeling Process, OWASP community content used with attribution.

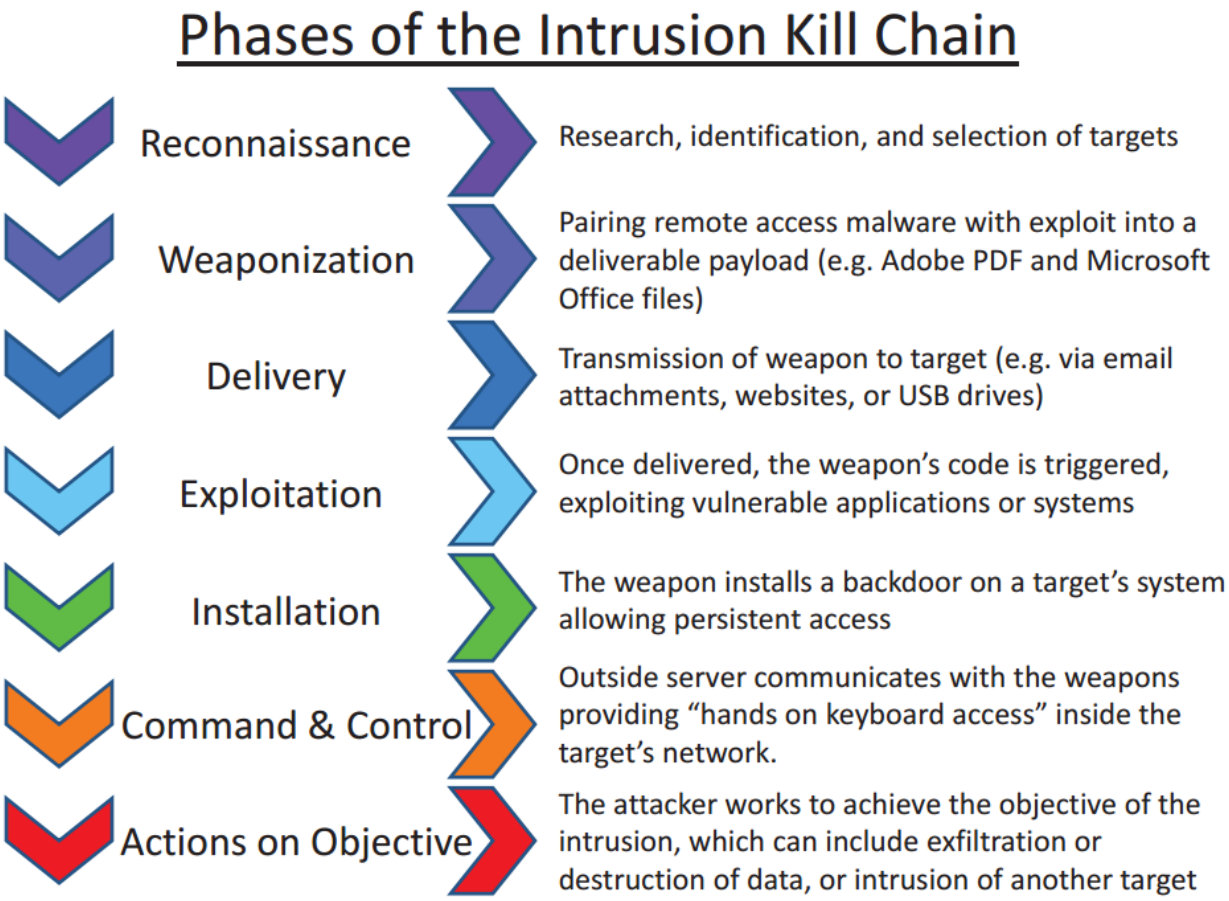

Figure. The intrusion kill chain, the seven-stage model this handbook's layer-transition sequence diagrams (Figure 3 above, and the multi-layer chains in Part 2 and Part 4) adapt from network stages to Web3 layer transitions. Source: Wikimedia Commons, U.S. Senate Committee on Commerce, Science, and Transportation, public domain (U.S. government work).

3.4 Risk Evaluation and Mitigation Mapping

Once a scenario is mapped, score it and route it to a control before moving to the next: an unscored threat model is a list of scary stories, not a work plan. Score likelihood and impact independently (High, Medium, or Low is enough resolution for a workshop), and always score impact using the worst realistic state the attack lands in, not the state the system happens to be in on the day of the workshop.

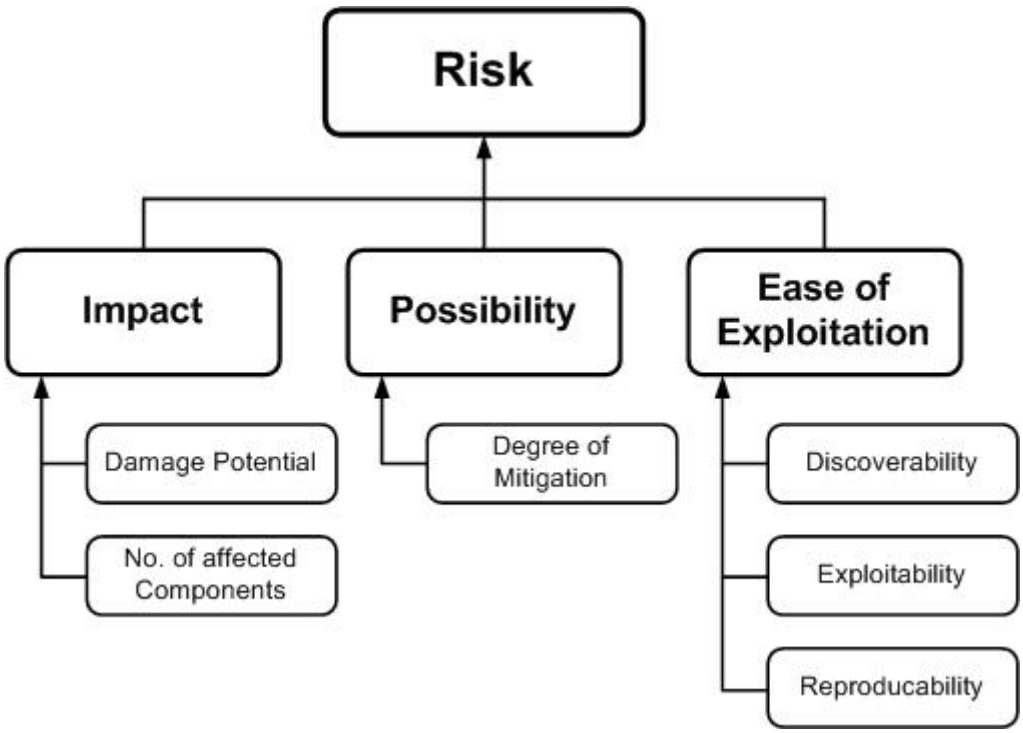

Figure. The OWASP risk-factor model prevents severity from collapsing into one intuition: threat agent and vulnerability properties determine likelihood, while technical and business consequences determine impact. For irreversible Web3 transfers, business impact may dominate even when observed frequency is modest. Source: OWASP Threat Modeling Process, OWASP community content used with attribution.

Scenario	Layer(s)	Likelihood	Impact	Priority	Primary mitigation	Owning handbook
Cloned front end alters calldata (Bybit pattern)	dApp, wallet	Medium	High	Critical	Human-readable calldata decoding at signing; SRI on wallet-connect scripts	01
Fake-interview malware plants a wallet stealer	User	High	High	Critical	Sandboxed interviews; no credential entry outside managed devices	05
DNS hijack redirects users to a phishing clone	dApp, infrastructure	Medium	High	High	Registry lock, DNSSEC, domain monitoring	02

Scenario	Layer(s)	Likelihood	Impact	Priority	Primary mitigation	Owning handbook
Insider pushes unauthorized contract upgrade	Infrastructure, contract	Low	High	Medium	Multi-party approval, timelock, signed provenance	06, SCSVS-ARCH
RPC provider outage strands users mid-transaction	Node, infrastructure	Medium	Medium	Medium	Multi-provider RPC failover	07

Table 6. A worked risk register. Priority is not a mechanical product of likelihood and impact: a Critical mitigation gets built even at Medium likelihood if the impact is unrecoverable fund loss. Part 3's TTP matrix supplies the detection and mitigation detail this table only summarizes.

Key controls for Part 1

- Treat every attack as a potential chain across the six layers (user, wallet, dApp, contract, node, infrastructure) before assuming it is contained to one.
- Use the OWASP Web3 Attack Vectors Top 15 as an unranked awareness checklist, assign an owner to every WA01 to WA15 vector, not a severity order.
- Cross-reference this handbook's TTP catalog to ATT&CK, AADAPT, WA, and SCWE identifiers rather than inventing parallel taxonomy.
- Run STRIDE across all six layers, not just the application boundary, and require an explicit reason to leave any cell blank.
- Score risk on worst-realistic-state impact, and route every scored scenario to a named control and an owning handbook before closing the workshop.

Part II: Tactics and Techniques by Layer

Part 2 works through the Web3 stack one layer at a time: the person holding the keys, the wallet that holds them, the dApp front end that requests a signature, the smart contract that executes it, the node and RPC (remote procedure call) path that relays it, and the infrastructure everything else runs on. Each chapter opens by aligning its layer to the relevant vectors from the OWASP Web3 Attack Vectors Top 15 (WA01 through WA15), works through the mechanics with a detection signal and a mitigation for every named technique, and closes with a technique catalog. The `T<chapter>.<sequence>` labels in this Part (for example, `T4.001`) are **chapter-local editorial locators**, not stable interchange identifiers. Part III assigns the canonical, never-reused `W3TTP-<LAYER>-<NNN>` identifiers to curated techniques. A machine-readable release must publish an explicit locator-to-canonical-ID crosswalk; consumers must not infer equivalence merely because two suffixes happen to match.

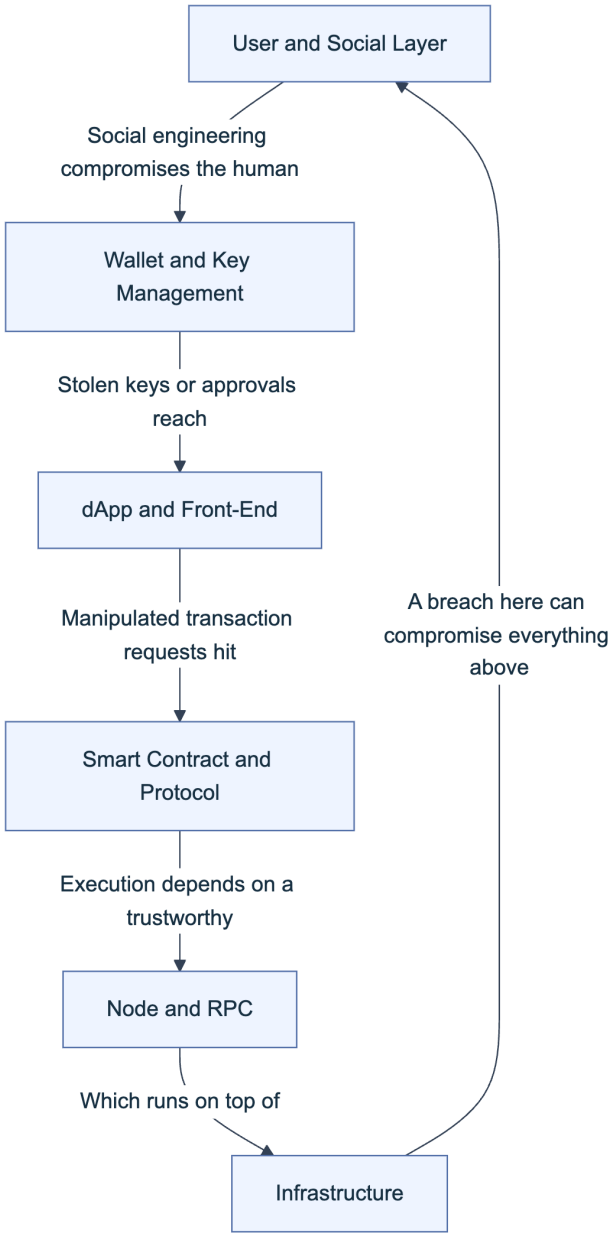

Figure 1. The six layers this Part covers, in the order a transaction actually flows. The feedback edge from Infrastructure back to User matters: a compromised build pipeline or a breached developer laptop at the infrastructure layer routinely becomes the root cause of a wallet- or dApp-layer incident, which is why Chapters 4 through 9 cross-reference each other rather than treating each layer as isolated.

4. User and Social Layer

The user and social layer is where most Web3 losses start, not in a contract's bytecode but in a message, a call, or a form that convinces a person to act against their own interest. Attacks here rarely touch code; they touch trust, urgency, and a target's willingness to believe a counterparty is who it claims to be.

4.1 Top 15 Alignment: WA05, WA08, WA09, WA11

Four Top 15 vectors describe attacks that require no code vulnerability at all: the target's own judgment is the exploited surface. WA05 (fake interviews), WA08 (phishing), WA09 (romance and investment fraud), and WA11 (physical coercion) share a common defensive posture: because there is no patch for a social engineering technique, controls have to intervene at the process or behavioral layer rather than the code layer. That means isolation of untrusted inputs (never run a stranger's code on a key-holding device), verification habits (confirm identity out of band before acting on a request), and organizational policy (support never initiates contact, recruiters never require pre-interview code execution). The OWASP Web3 Attack Vectors Top 15 groups these under its social-engineering-driven vectors precisely because pattern-matching detection (malicious signatures, static analysis) cannot see them; only behavioral and process controls can.

4.1.1 WA05: Fake Interview and Video Call Social Engineering

North Korea-aligned threat actors run a well-documented recruiting lure against Web3 and traditional software developers. A fake recruiter reaches out on LinkedIn or a developer job board with an unusually well-paid remote role, moves the conversation to a "technical interview," and asks the candidate to run a take-home coding project or install a video-conferencing tool for the call. The project's dependency tree or the installer carries an infostealer loader, tracked across multiple campaigns under names including BeaverTail and InvisibleFerret, that the security community has documented under the umbrella term Contagious Interview and, separately, DeceptiveDevelopment (Palo Alto Networks Unit 42, ESET WeLiveSecurity). Once installed, the loader harvests browser-extension wallet keystores, session cookies, and SSH keys from the candidate's machine.

Detection signals include an unsolicited high-compensation offer that moves unusually fast to a technical stage, and any request to execute unfamiliar code or install an unfamiliar binary before an interview has been verified through a second channel. The mitigation is procedural: run every candidate assignment inside a disposable, network-isolated virtual machine or container with no wallet extensions, no production credentials, and no SSH keys present, and treat any interview process that requires bypassing that isolation as disqualifying in itself. Employers should adopt the same rule in reverse (candidates should never receive an unsigned, unreviewed executable as a "test").

4.1.2 WA08: Phishing and General Social Engineering

General phishing covers the high-volume channel: email, SMS, and direct messages on Discord, Telegram, or X that carry a malicious link or a request for a credential, seed phrase, or signature. Web3-specific variants include fake wallet-support tickets, fake exchange security alerts demanding an urgent login, and QR-code phishing ("quishing") at conferences where a scanned code opens a spoofed wallet-connect page instead of the legitimate one. Because these campaigns

operate at scale, defenders benefit from treating them statistically rather than case by case; the FBI's Internet Crime Complaint Center tracks cryptocurrency-related fraud losses in the billions of dollars annually across these categories (FBI IC3 annual reports).

Detection signals include look-alike domains (character substitution, added hyphens, wrong top-level domain), urgency language ("act within 24 hours or lose access"), and any inbound message that requests a seed phrase, private key, or signature under time pressure. Mitigation combines technical controls, hardware-backed multi-factor authentication (WebAuthn/FIDO2) that phishing pages cannot relay, canonical bookmarked links instead of search-result or DM links, and email-authentication monitoring (DMARC, SPF, DKIM) on the organization's own domains so its brand cannot be spoofed as easily.

4.1.3 WA09: Romance, Investment, Impersonation, Recovery and Pig Butchering Scams

Pig butchering, translated from the Chinese *sha zhu pan*, describes a long-con fraud in which the attacker builds a relationship, romantic or purely social, over weeks or months before introducing a "can't miss" investment opportunity on a platform the attacker controls. The victim's early deposits appear to grow, sometimes with fabricated withdrawals to build confidence, until a large deposit is requested and the platform vanishes or blocks withdrawal behind a fabricated "tax" or "compliance fee." A parallel pattern, "recovery scams," targets people who have already lost funds by posing as a recovery service and charging an upfront fee to reverse an irreversible loss. The FBI's 2023 IC3 report placed investment fraud, dominated by this pattern, as the single largest category of reported cryptocurrency losses, and Chainalysis tracks pig-butchering infrastructure as an organized, often forced-labor-staffed industry concentrated in parts of Southeast Asia (Chainalysis Crypto Crime research).

Detection signals include a new personal contact who steers conversation toward an unfamiliar trading app outside official app stores, and any platform that shows paper gains but blocks withdrawal without an additional payment. Mitigation is largely educational at the individual level and typological at the institutional level: exchanges and custodians can screen outbound transfers against known scam-cluster address lists and flag the deposit-then-large-withdrawal pattern characteristic of this fraud.

4.1.4 WA11: Wrench Attacks and Physical Coercion

A "wrench attack" (the term borrows from the xkcd 538 "security" comic, which observed that a five-dollar wrench defeats cryptography no algorithm can) is physical coercion used to extract a seed phrase, a private key, or a signed transaction directly from a person. 2025 saw a documented wave of such attacks in France targeting cryptocurrency holders and their families, including the kidnapping of a Ledger co-founder and his partner in January 2025, during which the attackers mutilated a victim to pressure a ransom payment; both were later rescued and multiple suspects

were arrested. Chainalysis and other blockchain intelligence firms began tracking wrench attacks as a distinct, rising 2025 trend, driven partly by public on-chain wealth signals and social-media self-disclosure of holdings (Chainalysis blog).

Detection is largely about exposure reduction rather than technical monitoring: the risk indicator is a person's own public disclosure of wealth (leaderboard rankings, conference talks naming a personal net worth, doxxable social accounts linked to a known address). Mitigation includes operational security around holdings disclosure, multisignature or time-locked withdrawal schemes that cannot be completed under immediate duress, decoy wallets holding a plausible but limited balance, and duress PINs on hardware wallets that open a limited-balance account under coercion. Personal physical security planning, covered in the Web3 Operational Security Handbook (11), is the primary control for individuals at elevated risk.

4.2 Phishing and Social Engineering

Beyond the WA08-specific channel list, phishing against Web3 targets follows patterns worth cataloging by delivery channel because detection tooling differs by channel. Spear-phishing against protocol team members is reconnaissance-driven: an attacker researches a target's role (treasury signer, deployer key holder, on-call engineer) from public GitHub commits, conference talks, or LinkedIn before crafting a tailored lure, often a fake job offer that doubles as WA05 reconnaissance or a fake urgent bug report requiring the target to run a "reproduction" script. OAuth consent phishing, where a victim is tricked into granting a malicious application scoped access to an email or cloud account rather than typing a password, evades traditional credential-phishing detection entirely because no password ever changes hands.

Channel	Typical payload	Detection surface	Primary control
Email	Malicious link, attached macro document	DMARC/SPF/DKIM failures, sender domain age	Email authentication enforcement, attachment sandboxing
SMS/voice ("smishing"/ vishing)	Urgent account-verification link or callback number	Sender number reputation, spoofed caller ID	Hardware-token 2FA that ignores SMS-based codes
Discord/ Telegram DM	Fake support, fake collaboration/investment offer	New account age, mismatched role badges	Server-level DM restrictions, verified-bot allowlists
X/social DM or reply	Fake giveaway, fake verification form	Reply-spam pattern, mismatched handle history	Platform reporting, brand-monitoring services
OAuth consent screen	Malicious app requesting broad account scope	Unusual app name/publisher, excessive scope request	Periodic connected-app audits, least-privilege OAuth scopes

Every row in this table converges on the same structural mitigation: verify identity and intent through a channel the attacker does not control, and never let time pressure substitute for that verification.

4.3 Impersonation and Support Scams

Impersonation attacks exploit the gap between a brand's visual identity and its actual communication channels. A fake support account replies to a user's public complaint on X or Discord before the legitimate team can, offering to "help" through a DM that leads to a credential or seed-phrase harvest; because the fake account often mimics the real support handle closely (an added underscore, a swapped letter), victims rarely notice. A related and higher-impact pattern is account takeover of an already-verified, high-follower account: the July 2020 mass compromise of prominent X (then Twitter) accounts, including major public figures and companies, to post a cryptocurrency giveaway scam remains the reference incident for how much a single hijacked "trusted" account can extract in a short window, reported by Twitter's own post-incident update at the time as netting over $100,000 in Bitcoin within hours across more than a hundred compromised accounts.

Homoglyph and punycode domains extend the same trick to URLs: a domain that renders visually identical to a legitimate one using Cyrillic or other Unicode look-alike characters is registered and used for phishing pages that copy the real site pixel for pixel. Detection signals include any support contact that originates as an unsolicited DM (legitimate support teams almost universally require the user to open a ticket first, not the reverse), a verified badge on an account whose recent posting behavior has abruptly changed, and browser-rendered domains that look correct but fail a punycode decode check. Mitigation combines platform-level account-recovery hardening (hardware-token 2FA on official social accounts, so a phished password alone cannot take them over), brand-monitoring services that flag look-alike domain registrations, and a hard organizational policy, published where users can find it, that support will never initiate contact or request a seed phrase under any circumstance.

4.4 Technique Catalog and TTP IDs

The table below assigns TTP IDs to every technique named in this chapter. Detection and mitigation columns are deliberately compressed here; the full reasoning for each sits in the corresponding subsection above. Part III's TTP Matrix (Chapter 10) reproduces these rows alongside every other chapter's catalog for cross-layer correlation.

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T4.001	Fake interview / trojanized take-home lure	WA05	Unsolicited high-pay offer moving fast to code execution	Disposable, network-isolated VM for all candidate code
T4.002	Mass and spear phishing (email/SMS/social)	WA08	Look-alike domain, urgency language, unexpected 2FA prompt	Hardware-token 2FA, canonical bookmarked links

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T4.003	Romance/ investment long-con (pig butchering)	WA09	New contact steers to unfamiliar trading platform	User education, exchange-side scam-cluster screening
T4.004	Wrench attack / physical coercion	WA11	Public wealth disclosure, unusual in-person contact	OPSEC on holdings, duress wallets, time-locked withdrawal
T4.005	Fake support / help-desk impersonation	WA08	Support contacts user first via DM	Published "support never DMs first" policy
T4.006	Hijacked/ verified-account impersonation and giveaways	WA08	Abrupt posting-pattern change on a known account	Hardware 2FA on official social accounts, brand monitoring

5. Wallet and Key Management

The wallet layer is where a social-engineering success becomes a financial loss: everything in Chapter 4 exists to reach the keys or the signature this chapter protects. A wallet compromise is rarely detectable by the victim in real time, because the attacker's goal is a single, fast, irreversible transfer.

5.1 Top 15 Alignment: WA01, WA03, WA04, WA06, WA14

Five Top 15 vectors converge on the wallet: WA01 (multisig hijacking), WA03 (private key compromise), WA04 (drainer malware and drainer-as-a-service), WA06 (UI/UX spoofing and approval phishing), and WA14 (wallet software and extension compromise). What unites them is the signing moment: every one of these techniques ends with the victim's wallet producing a valid signature over attacker-chosen data, whether through stolen keys, a manipulated display, or a multisig quorum individually deceived. Defenses cluster the same way, around making the signing moment trustworthy: clear signing (showing a human-readable summary of what is actually being signed, not raw hex), transaction simulation before signing, and hardware isolation of the key itself.

5.1.1 WA01: Multisig Hijacking

Multisignature (multisig) wallets require several independent signers to approve a transaction, which should make a single compromise insufficient. Attackers instead target the thing every signer trusts in common: the interface that shows them what they are signing. The February 2025

Bybit incident, at the time the largest cryptocurrency theft on record at roughly $1.4 to $1.5 billion in ether, worked exactly this way: attackers compromised a developer machine tied to Safe{Wallet}'s front-end infrastructure and altered the transaction-proposal interface so that Bybit's cold-wallet signers saw a legitimate-looking transfer while the actual calldata routed funds to attacker-controlled addresses, a blind-signing failure at organizational scale attributed by the FBI to North Korea's TraderTraitor group (FBI IC3 advisory, February 2025). WazirX suffered a comparable multisig compromise in July 2024, losing roughly $230 million when signers approved a transaction that silently changed the multisig's underlying implementation contract.

Detection signals include any mismatch between a wallet's independent transaction simulation and what the front end displays, and any unexpected change to a multisig's implementation or logic contract address. Mitigation requires hardware wallets that clear-sign the actual calldata (not a UI-rendered summary the hardware device cannot itself verify), independent transaction-simulation tooling run by each signer before approving, out-of-band verification of the calldata hash among signers over a channel the compromised UI cannot influence, and combining signature thresholds with time locks so a malicious approval can be caught and reversed before execution.

5.1.2 WA03: Private Key Compromise

Direct key theft covers infostealer malware families (commonly reported strains include RedLine, Lumma, and Raccoon-class stealers) that scan a browser's local storage for extension-wallet keystores, physical theft or coercion of a written seed phrase, and cloud-backup misconfiguration where a user photographs or screenshots a seed phrase that then syncs to an iCloud or Google Drive account with its own separate compromise surface. Clipboard hijackers are a distinct but related technique: malware that monitors the system clipboard and silently swaps a copied wallet address for an attacker-controlled one immediately before the victim pastes it into a transfer field.

Detection depends on endpoint visibility: endpoint detection and response (EDR) tooling that alerts on an unrelated process reading browser-extension local storage, or on a process registering a clipboard-change hook, catches both patterns before the theft completes. Mitigation is architectural: hardware wallets keep the private key off any general-purpose, internet-connected device entirely; passphrase-protected ("25th word") wallets defeat a stolen seed phrase alone; and a dedicated, minimal-software device used only for signing removes most of the attack surface infostealers rely on.

5.1.3 WA04: Drainer Malware and Drainer-as-a-Service (DaaS)

Drainer-as-a-service describes a criminal business model: a developer builds a reusable "drainer" script, a piece of front-end JavaScript that connects to a victim's wallet and requests a single high-value signature, then rents it to affiliates for a percentage of everything stolen, commonly reported in the 10 to 30 percent range by researchers tracking kits such as Inferno Drainer and Pink Drainer. The affiliate handles distribution (a phishing site, a fake airdrop claim, a

compromised Discord server) while the kit handles the technical work of constructing the malicious approval or `permit` signature and sweeping the victim's assets the instant it is signed, often within the same block.

Detection signals include a sudden spike in `Approval` events granted to a newly deployed or previously unseen spender contract, and browser-extension phishing-site blocklists (maintained by services such as ScamSniffer and individual wallet vendors) flagging the connecting domain before the wallet-connect handshake completes. Mitigation combines periodic approval revocation using tooling such as Revoke.cash so stale, unused approvals cannot be exploited later, wallet-level transaction simulation that previews the net asset change before signing, and browser extensions that check outbound connection domains against a maintained drainer blocklist.

5.1.4 WA06: UI/UX Spoofing and Approval Phishing

At the wallet layer, UI/UX spoofing means the signing dialog itself lies: a wallet or a connecting dApp shows a summary ("Connect to MyDeFi") that does not match the underlying request ("grant unlimited token transfer approval to this address"), or a spoofed wallet-verification page asks a user to "reconnect" and sign a message that turns out to be a live transaction rather than the harmless signature it claims to be. This is the wallet-side half of a technique that also appears from the dApp side in Section 6.1.2; the distinction matters for defenders because the fix differs, wallet vendors must render clear-signed summaries accurately, while dApp operators must not construct requests designed to obscure their true effect. Front-end integrity controls (SRI, CSP) that prevent a spoofed clone site from existing in the first place are covered in depth in the CDN and Front-End Supply Chain Security Handbook (01).

5.1.5 WA14: Wallet Software, Extension and App Compromises

This vector covers the wallet application itself, not the site connecting to it. Trojanized wallet apps distributed through unofficial app stores or sideloaded outside Google Play and the Apple App Store impersonate popular wallets such as MetaMask or Trust Wallet closely enough in name and icon to catch a search-driven install; security researchers at firms including ESET and Kaspersky have repeatedly documented waves of such fake wallet apps reaching official app stores before takedown. A second pattern targets the legitimate extension's update mechanism: if an attacker compromises the developer account or build pipeline that ships extension updates, every installed copy receives the malicious version automatically, an attack class that overlaps directly with the supply-chain techniques in Section 6.1.1.

Detection signals include a wallet app with an unusually low download count and recent publication date impersonating an established brand, and any browser-extension update that changes requested permissions without a corresponding, verifiable release note. Mitigation includes installing wallet software only from the vendor's own linked, verified store listing, checking

extension publisher identity and permission scope on every update, and, for organizations, pinning extension versions and re-approving updates through a review step rather than accepting automatic updates silently.

5.2 Key Theft and Malware

Malware targeting wallets follows a predictable kill chain regardless of the specific family involved: initial access (a phishing lure, a trojanized installer, a fake interview package), execution on the target's machine, credential harvesting (browser-extension local storage, keychain files, clipboard content), and exfiltration to an attacker-controlled server before the theft itself occurs, often with the actual transfer executed by the attacker's own infrastructure rather than the malware itself. Remote access trojans (RATs) extend this further by giving the attacker interactive control, letting them wait for the victim to unlock a hardware wallet or type a passphrase before triggering a live transaction.

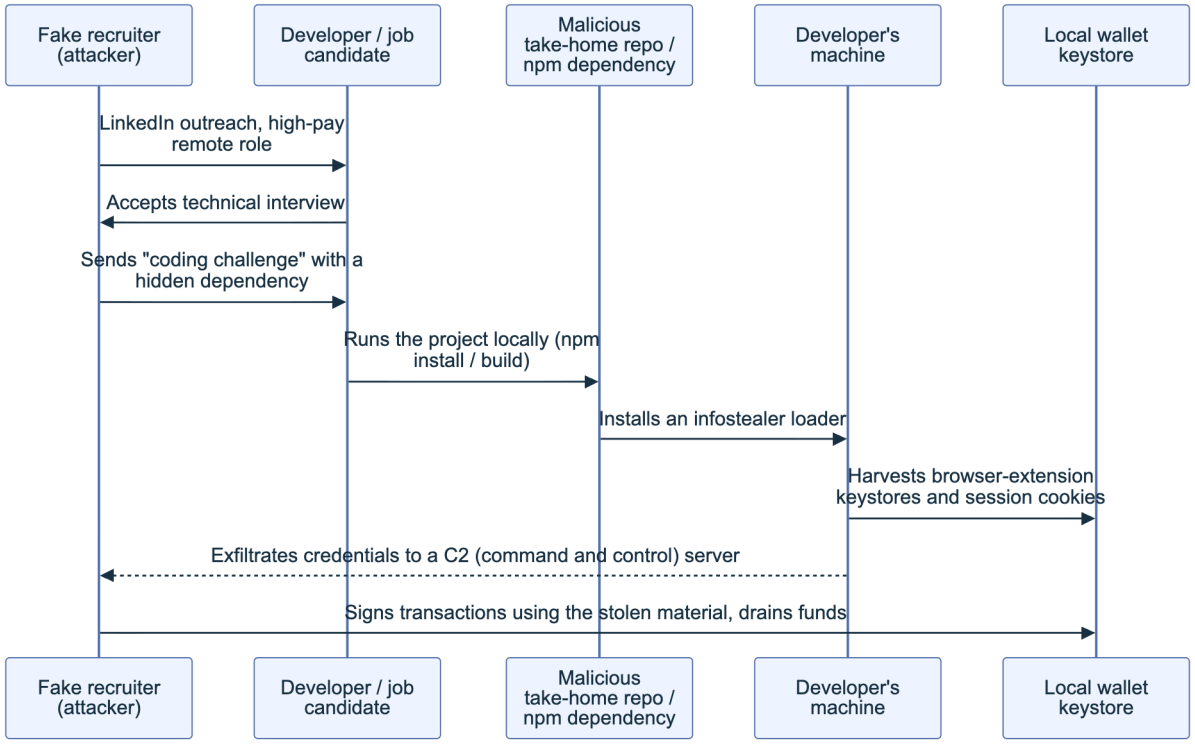

Figure 2. The fake-interview-to-key-theft chain (WA05 into WA03), the mechanism behind the Contagious Interview/Deceptive-Development campaigns discussed in Section 4.1.1. The same pattern recurs at organizational scale as nation-state infiltration in Section 9.1.4, where the "developer" is a placed insider rather than an interview candidate.

The chain-level defense is isolation at every hop: sandboxed execution for untrusted code (breaks the Repo-to-Host edge), EDR alerting on keystore and clipboard access (breaks the Host-to-Wallet edge), and network egress monitoring for unexpected outbound connections from a development machine (breaks the Host-to-C2 edge). No single control is sufficient; the chain is only as strong as its weakest unbroken edge.

5.3 Approval and Signature Abuse

Token approvals exist to let a contract move tokens on a user's behalf without a signature on every single transfer: ERC-20's `approve` and `increaseAllowance`, ERC-721 and ERC-1155's `setApprovalForAll`, and the gasless variants `permit` (EIP-2612) and Uniswap's Permit2. Every one of these mechanisms grants standing authority, sometimes unlimited, that persists until explicitly revoked, which makes a single signed approval as dangerous as a direct transfer if it is signed unknowingly. The core defensive problem is that a `permit` signature is an off-chain EIP-712 typed-data structure, and a poorly implemented wallet can render it as an opaque hex blob instead of a decoded, human-readable summary.

```
# What the user should be shown (clear-signed EIP-712 summary)
Grant approval to: 0xA1b2...F00d (labeled "Unknown contract, 3 days old")
Token: USDC
Amount: UNLIMITED
Expiry: none

# What a raw, undecoded signing request looks like to the user
0x19010a2f8c... (416 hex characters, no human-readable summary)
```

A wallet that only ever shows the second form cannot support informed consent regardless of how careful the user is; this is the mechanical root of most approval phishing.

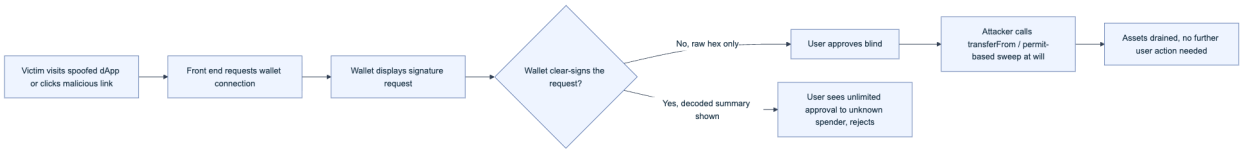

Figure 3. Approval phishing succeeds or fails at the clear-signing decision point. Detection tooling (transaction simulators, approval-scanning browser extensions) and an emerging clear-signing metadata standard for structured display of contract interactions (ERC-7730 and related efforts) both target this exact junction.

Detection signals include any approval request for an unlimited amount rather than a scoped one, and any spender contract with no verified source code or a deployment age of hours rather than months. Mitigation includes preferring wallets and interfaces that clear-sign `permit` and `setApprovalForAll` requests with a decoded summary, scoping approvals to the exact amount needed rather than accepting a dApp's default unlimited request, and running periodic approval audits with revocation tooling.

5.4 Wallet UI and Redress

"Redress" in a Web3 context is limited by design: blockchain transactions are final, so the primary lever is prevention at the signing moment rather than reversal afterward. Wallet UI design choices directly determine how much a user can actually understand before signing. Hardware wallets historically constrained this further with small screens that could show only a few lines of a transaction, forcing either truncation (hiding the very fields an attacker manipulates) or a trust handoff to the connected software wallet's rendering, which is exactly the trust the Bybit-style

multisig compromise abused. Industry initiatives, including Ledger's clear-signing push and the associated ERC-7730 metadata standard for describing how a contract's calldata should be rendered to a human, aim to let any wallet display a decoded, contract-author-defined summary rather than relying on each wallet vendor to reverse-engineer every protocol's calldata independently.

Where prevention fails, redress options are narrow and time-sensitive: rapid on-chain tracing and exchange notification (covered operationally in the EVM Forensics and DeFi Recovery Handbook, 04) to flag destination addresses before funds move further, and, for protocol-level incidents, a governance or multisig-controlled pause function if one exists and has not itself been compromised. Neither substitutes for prevention, and both depend on speed measured in minutes, not the hours a typical incident-response process assumes. Wallet vendors and dApp teams should treat the signing dialog as the single highest-leverage UI surface in the entire stack and invest accordingly: every ambiguity resolved there prevents an incident that no amount of downstream tracing fully recovers.

5.5 Technique Catalog and TTP IDs

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T5.001	Multisig hijacking via compromise d signer UI	WA01	Simulation-vs-display mismatch; unexpected implementation change	Independent simulation, hardware clear-signing, time-locked thresholds
T5.002	Private key compromise (infostealer, clipboard hijack)	WA03	EDR alert on keystore/clipboard access	Hardware wallets, passphrase protection, dedicated signing device
T5.003	Drainer-as-a-service kits	WA04	Spike in approvals to unfamiliar spender	Approval revocation, transaction simulation, drainer blocklists
T5.004	UI/UX spoofing of signing dialogs	WA06	Signing summary does not match underlying request	Clear-signing wallets, SRI/CSP on connecting front ends
T5.005	Malicious wallet extension or trojanized app	WA14	Low-download impostor app; permission scope change on update	Install only from vendor-verified listings, pin extension versions
T5.006	Seed-phrase phishing (fake recovery form)	WA03/WA08	Any form requesting a full seed phrase	User education, no legitimate wallet ever requests a seed phrase

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T5.007	Approval/ `permit` signature abuse	WA04/WA06	Unlimited-amount request to an unverified spender	Scoped approvals, clear-signed `permit` display
T5.008	Clipboard hijacker / address poisoning	WA03	Pasted address differs from copied source	Clipboard-monitoring EDR rules, always verify first/last characters

6. dApp and Front-End

The dApp and front-end layer is the delivery mechanism for everything above it: even a perfectly secure wallet and a perfectly audited contract can be defeated if the page constructing the transaction is compromised. This chapter summarizes the layer at the level a threat model needs; the CDN and Front-End Supply Chain Security Handbook (01) is the deep technical reference for every control named here.

6.1 Top 15 Alignment: WA02, WA06, WA10, WA13

WA02 (supply chain attacks), WA06 (deceptive interfaces), WA10 (rug pulls and token impersonation), and WA13 (DNS and routing hijacking) all converge on the same target: the code a browser executes when a user opens a dApp. What differs is the point of insertion, a poisoned dependency (WA02), a deliberately deceptive interface built by the protocol's own team (WA10), or a hijacked delivery path redirecting to attacker-controlled content (WA06 in its clone-site form, and WA13). A threat model for this layer has to cover all three insertion points, because hardening one does nothing for the others.

6.1.1 WA02: Supply Chain Attacks (npm, PyPI, OSS)

Front-end applications resolve to thousands of transitive dependencies from npm, PyPI, and other open-source registries, and any one of them is a potential insertion point. The reference incidents span a decade: the 2018 event-stream compromise inserted a Bitcoin-wallet-targeting payload into a popular npm package after a malicious actor gained trusted-maintainer access; the October 2021 ua-parser-js compromise pushed a cryptominer and password stealer through a hijacked maintainer account; and in December 2024 a compromised maintainer account published malicious versions of the `@solana/web3.js` package that attempted to exfiltrate private keys from any application that updated to the poisoned release, prompting an official advisory from the Solana Foundation and package registry. Security researchers also tracked a large-scale, self-propagating npm worm campaign in September 2025, publicized under the name Shai-Hulud, that stole publish credentials from compromised maintainers to republish itself into further packages.

```
// Illustrative pattern security teams grep for during dependency review,
// NOT a working payload: a postinstall hook that reaches out to a
// remote host during package installation is a red flag regardless
// of what it claims to do.
"scripts": {
  "postinstall": "node ./scripts/fetch-remote-config.js"
}
```

Detection signals include lockfile diffs that show a dependency version bump with no corresponding upstream changelog entry, and any `postinstall` or `preinstall` script that performs network activity. Mitigation includes pinning exact dependency versions and hashes rather than floating ranges, mirroring registries through a private proxy with a manual review gate for new versions, and verifying package provenance attestations where the registry supports them, all covered in depth in Handbook 01's supply-chain part.

6.1.2 WA06: Deceptive dApp Interfaces and Approval Phishing

From the dApp side, this vector covers spoofed clone sites (pixel-identical copies of a legitimate dApp hosted on a look-alike domain) and malicious search-engine advertisements that outrank the legitimate site for high-value brand terms such as a popular DEX (decentralized exchange) or wallet name, a pattern researchers have repeatedly documented against Uniswap, MetaMask, and other high-traffic Web3 brands. The wallet-side half of this vector, covering how a signing dialog itself can deceive, is detailed in Section 5.1.4 and 5.3; here the emphasis is on how the deceptive interface reaches the user in the first place. Detection and mitigation for the delivery path (SRI, CSP, ad-fraud monitoring, brand-protected search advertising) are covered in Handbook 01, Parts II and III.

6.1.3 WA10: Rug Pulls, Fake Airdrops and Token Impersonation

A rug pull is a front-end and contract-layer deception at once: the interface presents a token or protocol as legitimate while a hidden function (an unrestricted mint, an owner-only sell-blocking modifier that makes the token a honeypot, or a disguised liquidity-withdrawal path) lets the operator extract value from participants. The November 2021 Squid Game-themed token is the reference case for the pattern's speed: the token's price rose sharply on hype before its creators disabled selling for everyone but themselves and the price collapsed to near zero within minutes, widely reported at the time including by BBC News. Fake airdrops extend the same trick to a "claim" flow: a page invites a user to sign a message to receive free tokens, but the signed payload is actually an approval or a `permit` granting the attacker transfer rights, sweeping the wallet the moment it is signed, directly overlapping with the drainer techniques in Section 5.1.3.

Detection signals include a token contract with an unverified or unusually short source (or one that verifies but contains an owner-gated transfer restriction), and any "claim" transaction that a simulator shows as an approval rather than a token receipt. Mitigation includes simulating every

claim transaction before signing, checking token contract verification and ownership renunciation status, and treating any project's own marketing claims about audits as a starting point for verification, not a substitute for it.

6.1.4 WA13: DNS, Domain and Routing Infrastructure Hijacking

If an attacker controls the DNS record or the hosting path a dApp's domain resolves to, every integrity control on the page itself becomes moot, because the attacker serves an entirely different page. The August 2022 Curve Finance incident is the reference case: attackers compromised the registrar-level DNS settings for curve.fi and redirected the domain to a malicious clone that requested a token approval, resulting in reported losses on the order of hundreds of thousands of dollars from users who trusted the familiar URL. The December 2021 BadgerDAO incident took an adjacent path, compromising a Cloudflare API key to inject a malicious script directly into the legitimate front end without touching DNS at all, resulting in losses reported near $120 million. Both incidents illustrate the same lesson from different entry points: the delivery path between a correct domain name and a correct byte stream has multiple independent links, and each one needs its own control. The DNS and Hosting Security Handbook (02) covers registrar hardening, DNSSEC, and routing protections in full depth.

6.2 Front-End Compromise and Supply Chain

Synthesizing Sections 6.1.1 and 6.1.4, front-end compromise is best modeled as a chain with four independent links: the dependency graph (code level), the build pipeline (build level), the CDN and DNS (deploy level), and the browser runtime (runtime level), the same four-level model developed in Handbook 01's foundations part. A defender auditing a dApp's front-end integrity should walk all four links rather than stopping at the first one checked, because an attacker only needs one weak link and will find whichever one the defender skipped.

Front-end integrity checklist (full technical detail in Handbook 01): - Every third-party script pinned with Subresource Integrity (SRI) and served from an immutable, versioned URL. - A nonce-based or hash-based Content Security Policy (CSP) with `strict-dynamic` deployed and violation reporting enabled. - Dependencies pinned to exact versions and hashes; a software bill of materials (SBOM, in CycloneDX or SPDX format) generated on every build. - Build artifacts signed (Sigstore/Cosign or equivalent) and verified before deployment. - DNS records protected with registrar-level 2FA, a registry lock, and DNSSEC. - CDN and cloud API credentials scoped to least privilege and rotated on a defined schedule.

6.3 DNS, ENS, and Hosting Abuse

The Ethereum Name Service (ENS) layers a human-readable naming system on top of Ethereum addresses, and it introduces its own impersonation surface alongside traditional DNS. Homoglyph ENS names (registering a name that renders visually similar to a well-known one using look-alike characters or subtly different casing) exploit the same visual-trust gap as homoglyph domains. A second ENS-specific technique targets the reverse record: because a wallet interface may display a

resolved ENS name in place of a raw address to make a transaction "friendlier" to read, an attacker who controls a matching reverse record can make an unrelated address display as a trusted-looking name, a social-engineering assist rather than a technical exploit.

Traditional DNS abuse against dApp hosting includes registrar account takeover (often itself downstream of the phishing techniques in Chapter 4, since registrar credentials are a high-value phishing target) and expired-domain re-registration, where an attacker registers a domain a project has since abandoned but that remains linked from old documentation, social posts, or bookmarks, and serves malicious content from it. Detection signals include unexpected changes to authoritative nameservers or DNS records outside a change window, and any ENS reverse-record lookup that resolves to a name inconsistent with the address's known history. Mitigation includes registrar-level hardening (hardware 2FA, registry lock, monitoring via RDAP for unauthorized changes) and DNSSEC deployment, both covered in full in the DNS and Hosting Security Handbook (02).

6.4 Connection and Transaction Manipulation

Beyond the display and delivery attacks above, the connection between a wallet and a dApp is itself a manipulable surface. WalletConnect and similar bridging protocols establish a session with a defined permission scope; a malicious or compromised dApp can request an overly broad scope (persistent session, unlimited method access) that a user approves without reading, giving the dApp standing ability to request signatures well beyond the immediate interaction. Session-relay infrastructure is also a target: an attacker positioned on the relay path, or one who has phished a session-approval QR code at a physical event by displaying it on a compromised or spoofed screen, can hijack an active session rather than needing to compromise the wallet itself.

Detection signals include a WalletConnect session request with an unusually broad or unlimited scope, and any QR-code scan at a physical venue that does not resolve to the expected, officially verified dApp identity. Mitigation includes scoping sessions to specific methods and a defined expiry rather than accepting default broad grants, reviewing and revoking active sessions periodically, and using WalletConnect's own domain-verification API where the connecting wallet supports it to confirm the requesting dApp's registered identity before approving a session.

6.5 Technique Catalog and TTP IDs

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T6.001	npm/PyPI/ OSS dependency supply chain injection	WA02	Lockfile diff with no changelog; network activity in install scripts	Pinned hashes, private registry review gate, provenance verification

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T6.002	Deceptive dApp clone site / malvertising	WA06	Look-alike domain; ad ranking above official site	SRI/CSP, brand-protected search advertising
T6.003	Rug pull via disguised mint/owner function	WA10	Unverified source or owner-gated transfer restriction	Transaction simulation, contract verification checks
T6.004	Fake airdrop / token impersonation claim	WA10	Claim transaction simulates as approval, not receipt	Simulate every claim before signing
T6.005	DNS/ domain/ routing hijack of front end	WA13	Unexpected nameserver or DNS record change	Registrar 2FA, registry lock, DNSSEC
T6.006	CDN/build-pipeline compromise	WA02/WA13	Unauthorized change to CDN/API config outside change window	Least-privilege, rotated CDN and cloud credentials
T6.007	WalletConnect session hijack / over-broad scope grant	WA06	Session request with unlimited method scope	Scoped sessions, periodic session review and revocation
T6.008	Homoglyph ENS name / spoofed reverse record	WA06/WA13	Reverse-record name inconsistent with address history	Manual address verification, ENS name-history checks

7. Smart Contract and Protocol

The smart contract layer is the one layer the OWASP Web3 Attack Vectors Top 15 deliberately does not re-catalog in depth, because the Smart Contract Top 10 (SC Top 10) and Smart Contract Weakness Enumeration (SCWE) already own on-chain logic risk in exhaustive detail. This chapter's job is to map, not duplicate: it places contract-layer techniques into the same TTP structure as every other layer so a cross-layer chain (Chapter 10's matrix) can reference a contract bug alongside the social-engineering or infrastructure step that made it exploitable.

7.1 Mapping to SCWE and SC Top 10 (On-Chain Logic; Top 15 Focuses on Non-Contract Vectors)

The OWASP Web3 Attack Vectors Top 15 explicitly positions itself as the *non-contract* companion to the SC Top 10, which catalogs on-chain logic risk (reentrancy, access control, arithmetic, and related classes), and to the SCWE, which enumerates individual weaknesses within those classes at a granularity comparable to CWE (Common Weakness Enumeration) for traditional software. A threat model that only consults the Top 15 will therefore systematically under-cover contract logic; this chapter exists to close that gap by giving contract-layer techniques the same TTP treatment as every other layer, cross-referenced outward to SCWE rather than restating its content.

This handbook's chapter 7 technique	SC Top 10 category	Representative SCWE reference
Reentrancy (T7.001)	Reentrancy	SCWE entries for external-call-before-state-update patterns
Oracle manipulation (T7.002)	Price/oracle manipulation	SCWE entries for unvalidated external price feeds
Access control failure (T7.003)	Access control	SCWE entries for missing or misconfigured modifiers
Business logic/rounding error (T7.004)	Logic errors	SCWE entries for arithmetic and precision-loss weaknesses

Readers building a full contract-layer threat model should treat this table as a pointer, then work the SCSVS verification requirements and SCWE catalog directly for exhaustive coverage; this handbook does not restate either.

7.2 Reentrancy, Oracle, Access Control, Logic

These four classes account for the large majority of reported on-chain value loss and are worth naming together because they share a root cause: a function that trusts state or an external input at the wrong moment. **Reentrancy** occurs when a contract makes an external call before finalizing its own state update, letting the called contract call back in and act on stale state; the pattern dates to The DAO incident of 2016 and remains common in modern variants (cross-function and read-only reentrancy) despite being the best-understood class in the field. **Oracle manipulation** exploits a contract's reliance on a price or data feed that an attacker can move, often within a single transaction using a flash loan (Section 7.3); the Harvest Finance incident of October 2020 and numerous later incidents against protocols using spot-price-derived oracles illustrate the pattern. **Access control failure** covers missing or misconfigured permission checks, exemplified by the 2017 Parity multisig wallet incident, where a library contract's initialization function was left callable by anyone, letting an attacker become its owner and subsequently self-destruct it, freezing funds in every wallet depending on it. **Logic and rounding errors** are the catch-all for arithmetic that behaves correctly in the common case but produces an exploitable result at a boundary value, such as a division that rounds in the attacker's favor when repeated at scale.

```
// Illustrative defensive pattern (checks-effects-interactions),
// not exploit code: state is finalized before the external call.
function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount, "insufficient balance");
    balances[msg.sender] -= amount;              // effect, before interaction
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "transfer failed");
}
```

Detection for this class of bug leans heavily on static analysis and formal methods rather than runtime monitoring, since the exploit condition often exists from deployment. Mitigation includes the checks-effects-interactions ordering shown above (or a reentrancy guard modifier as a defense in depth), time-weighted or multi-source oracles instead of single-block spot prices, exhaustive access-control test coverage for every state-mutating function, and boundary-value testing (zero, one, and maximum-value inputs) for every arithmetic path.

7.3 Flash Loans and Economic Exploits

A flash loan lets a borrower draw an arbitrarily large, uncollateralized amount of capital for the duration of a single transaction, provided the loan is repaid with a fee before the transaction ends. That capability is economically neutral by itself, but it collapses the capital requirement for any attack that depends on temporarily moving a market or a governance vote, since the attacker never needs to own the capital, only to borrow, manipulate, extract, and repay atomically. The February 2020 bZx incidents were among the first widely analyzed flash-loan-funded oracle manipulations; the April 2022 Beanstalk incident used a flash loan to briefly acquire enough of the protocol's governance token to pass and execute a malicious proposal in a single transaction, extracting roughly $182 million; and the March 2023 Euler Finance incident, resulting in losses of roughly $197 million (subsequently returned after negotiation), combined a flash loan with a donation-based accounting flaw rather than direct price manipulation, illustrating that the technique generalizes beyond oracle attacks to any state a single atomic transaction can distort.

Incident	Date	Mechanism	Reported impact
bZx	Feb 2020	Flash-loan-funded spot price manipulation against an oracle	Low hundreds of thousands (USD)
Harvest Finance	Oct 2020	Flash-loan-funded price manipulation against a Curve pool oracle	~$24 million
Beanstalk	Apr 2022	Flash-loan-funded governance takeover, single-transaction proposal execution	~$182 million
Euler Finance	Mar 2023	Flash-loan-funded donation/accounting exploit	~$197 million (later returned)

Detection for flash-loan-funded attacks focuses on transaction-level anomaly patterns: a single transaction that borrows an unusually large amount, interacts with a governance or pricing function, and repays within the same transaction is a strong signal, and several mempool-monitoring and simulation services now flag this pattern before inclusion. Mitigation includes time-weighted average price (TWAP) oracles resistant to single-block manipulation, governance designs that require proposal and execution to occur in separate blocks (breaking the atomicity the attack depends on), and borrow-size circuit breakers on lending markets that supply flash loans.

7.4 Technique Catalog and TTP IDs

TTP ID	Technique	SC Top 10 category	Primary detection signal	Primary mitigation
T7.001	Reentrancy	Reentrancy	External call before state finalization in static analysis	Checks-effects-interactions, reentrancy guards
T7.002	Oracle manipulation	Price/oracle manipulation	Single-block spot-price dependency	TWAP or multi-source oracles
T7.003	Access control failure	Access control	Missing/misconfigured permission modifier	Exhaustive access-control test coverage
T7.004	Business logic/ rounding error	Logic errors	Boundary-value tests fail (0, 1, max)	Formal verification, boundary-value fuzzing
T7.005	Flash-loan-funded price manipulation	Price/oracle manipulation	Large atomic borrow-manipulate-repay pattern	TWAP oracles, borrow-size circuit breakers
T7.006	Flash-loan-funded governance takeover	Access control / governance	Proposal and execution in the same transaction/block	Time-separated proposal and execution windows

8. Node and RPC

The node and RPC (remote procedure call) layer is the plumbing between a wallet or dApp and the chain itself. It receives comparatively little attention in most Web3 threat models because it is often outsourced to a third-party provider, which is exactly why it deserves explicit coverage: outsourcing responsibility does not outsource risk.

8.1 RPC Abuse and Manipulation

Every wallet and dApp interaction with a blockchain passes through an RPC endpoint, a server that answers queries such as “what is this address’s balance” and relays signed transactions to the network. An attacker who controls the RPC endpoint a victim’s wallet is configured to use controls the wallet’s entire view of reality: it can report fabricated balances, omit pending or malicious transactions from a wallet’s activity feed, or silently redirect a submitted transaction’s broadcast. The most common delivery mechanism is a malicious “Add Network” or “Add Custom RPC” prompt on a phishing site, which many wallets accept with minimal friction because switching networks is a routine, low-perceived-risk action.

```
// Illustrative structure of a wallet_addEthereumChain request
// (EIP-3085). A mismatched chainId paired with a familiar-sounding
// chainName is the detection signal, not a working exploit.
{
  "chainId": "0x89",
  "chainName": "Polygon Mainnet",
  "rpcUrls": ["https://rpc.attacker-controlled.example"],
  "nativeCurrency": { "name": "MATIC", "symbol": "MATIC", "decimals": 18 }
}
```

Detection signals include a network-addition prompt whose `chainId` does not match the well-known value for the claimed network, and an RPC endpoint whose responses diverge from a second, independently queried provider for the same query. Mitigation includes wallets validating `chainId` against a maintained registry of known chains before accepting a custom network, users cross-checking balances against a second RPC provider or a block explorer before trusting a discrepancy, and organizations pinning wallet configurations to a small, vetted set of RPC providers rather than accepting arbitrary custom endpoints.

8.2 Node Compromise and Consensus (Where Applicable)

Direct compromise of the consensus layer itself, rather than the RPC interface in front of it, is a narrower but higher-impact concern that applies mainly to smaller or less-decentralized networks rather than to the largest proof-of-stake chains, where the economic cost of acquiring sufficient stake is prohibitive. Proof-of-work chains with modest hash rate remain the clearest example: Ethereum Classic suffered multiple documented 51 percent (majority hash rate) attacks in January 2019 and again in August 2020, each enabling a deep chain reorganization that let the attacker double-spend already-confirmed transactions on centralized exchanges. A related but distinct technique, the eclipse attack, does not require majority hash rate at all: it isolates a single victim node by surrounding its peer connections with attacker-controlled nodes, feeding it a fabricated view of the network without needing to attack the broader consensus.

A newer variant of this concern applies to rollups and other layer-2 networks that rely on a centralized or lightly decentralized sequencer to order transactions: a compromised or malicious sequencer can reorder, delay, or censor transactions even without any consensus-layer attack in

the traditional sense, an architectural risk actively discussed as rollups move toward decentralized sequencing. Detection signals include an unexpected deep chain reorganization reported by monitoring infrastructure, and a node whose peer set shows unusually low diversity in IP ranges or autonomous system numbers. Mitigation includes requiring a higher confirmation count for high-value transactions on lower-hash-rate chains, running validating nodes with diverse, monitored peer connections to reduce eclipse risk, and, at the protocol design level, pursuing sequencer decentralization or fraud-proof mechanisms that bound how much damage a compromised sequencer can cause.

8.3 Technique Catalog and TTP IDs

TTP ID	Technique	Primary detection signal	Primary mitigation
T8.001	Malicious "Add Network"/RPC endpoint spoofing	`chainId` mismatch on network-addition prompt	Wallet-side chain ID validation against known registry
T8.002	RPC response manipulation (fake balance/gas)	Divergence from a second, independent RPC provider	Cross-check balances via block explorer or second provider
T8.003	MEV sandwich/front-running via mempool visibility	Abnormal price impact around a pending transaction	Private mempools/relays, slippage limits
T8.004	Eclipse attack on node peer connections	Low peer IP/ASN diversity	Diverse, monitored peer connections
T8.005	51 percent/consensus reorganization attack	Unexpected deep chain reorganization	Higher confirmation thresholds on lower-hash-rate chains

9. Infrastructure

Infrastructure is the layer everything else assumes is trustworthy: the exchange backend, the cloud account, the registrar, and the people with administrative access to all three. A breach here rarely stays contained to this layer, which is why several techniques in this chapter reappear as the root cause of incidents already discussed at other layers.

9.1 Top 15 Alignment: WA07, WA12, WA13, WA15

WA07 (centralized exchange and Web2/2.5 infrastructure breaches), WA12 (insider threats), WA13 (DNS and routing hijacking, revisited here from the infrastructure-operator side rather than the dApp-consumer side covered in Chapter 6), and WA15 (nation-state infiltration) share

an operational, rather than purely technical, defensive posture: identity and access management, vendor and third-party risk management, and personnel security controls do more work here than any code-level fix.

9.1.1 WA07: Centralised Exchange and Web2/2.5 Infrastructure Breaches

Exchanges and other centralized custodians hold value that concentrates attacker interest, and their breaches have produced some of the industry's largest losses: Mt. Gox in 2014 (roughly 850,000 bitcoin, a substantial share never recovered), Coincheck in January 2018 (roughly $530 million in NEM tokens, attributed to a hot-wallet compromise), and the February 2025 Bybit incident already detailed in Section 5.1.1, which is worth re-reading from this chapter's angle: the root cause traced to a compromised Web2/2.5 developer workstation inside Safe{Wallet}'s infrastructure, not a flaw in Bybit's own systems, illustrating how a vendor's Web2 infrastructure breach becomes every downstream customer's Web3 incident. FTX's November 2022 incident, occurring during the exchange's bankruptcy filing, involved an unauthorized on-chain drain of several hundred million dollars whose attribution (external attacker versus insider access) remained disputed in subsequent reporting.

Detection signals include hot-wallet balance changes inconsistent with recorded customer withdrawal activity, and unusual administrative access patterns on exchange back-office systems. Mitigation includes cold-storage majority custody with a minimal, closely monitored hot-wallet balance, multi-party computation (MPC) or hardware-backed signing for any hot-wallet transaction, and treating every third-party infrastructure vendor's own security posture as part of the exchange's own attack surface, not a separate concern.

9.1.2 WA12: Insider Threats and Collusive Abuse

An insider with legitimate access bypasses every perimeter control built to keep external attackers out. The risk spans a spectrum from a single disgruntled employee exfiltrating credentials on their way out, to coerced or bribed staff providing access under duress or payment, to fully collusive schemes where an employee actively partners with an external actor, for example bypassing know-your-customer (KYC) controls to onboard a laundering-focused account. Detection relies on behavioral analytics (access outside normal patterns, data exfiltration volume anomalies) more than signature-based tooling, since an insider's individual actions are each, in isolation, legitimate. Mitigation includes least-privilege access provisioning, mandatory access review on role change, separation of duties for any high-value action (no single employee able to both approve and execute a large transfer), and prompt, verified credential revocation on termination, covered operationally in the Employee Lifecycle Security Handbook (03) and the Hiring, Remote Work, and Insider Threat Handbook (05).

9.1.3 WA13: DNS, Domain and Routing Hijacking

From the infrastructure operator's side, this vector is about the controls a team runs on its own registrar and network accounts rather than the front-end consequences covered in Section 6.1.4 and 6.3. Registrar account security (hardware-backed 2FA, a registry lock requiring an out-of-band step to change nameservers), continuous monitoring of authoritative DNS records through the Registration Data Access Protocol (RDAP) for unauthorized changes, and, at the network layer, Resource Public Key Infrastructure (RPKI) to reduce the risk of Border Gateway Protocol (BGP) route hijacking, are the operational controls that prevent the incidents Chapter 6 describes from the victim's perspective. The DNS and Hosting Security Handbook (02) is the full reference for implementing each of these.

9.1.4 WA15: Nation-State Infiltration via Fake Hiring and Malicious OSS Contributions

This vector inverts the direction of Section 4.1.1's fake-interview technique: rather than a fake recruiter targeting a real developer, a nation-state actor poses as a real developer to get hired inside a target organization. United States Department of Justice indictments issued across 2023 and 2024 detailed a sustained North Korean scheme placing operatives in remote developer roles at Western companies using stolen or fabricated identities, with wages funneled back to fund the regime's weapons programs; security awareness firm KnowBe4 publicly disclosed in 2024 that it had unknowingly hired one such operative, who began installing malware within minutes of receiving a company laptop, an incident notable for how quickly the placed insider moved from access to action. A separate but related technique targets open-source projects directly rather than employers: the XZ Utils backdoor involved an actor building maintainer trust in a widely used compression library over more than two years before inserting a backdoor into official release tarballs, discovered in March 2024 by a Microsoft engineer investigating unrelated performance anomalies.

Detection signals include a remote hire whose identity documents, video-call behavior, or payment-routing requests show inconsistencies (multiple job offers accepted simultaneously, a request to route payment through a third party, reluctance to appear on unscripted video calls), and, for open-source projects, a sudden escalation in a long-time contributor's push for merge access or release authority. Mitigation includes identity verification during hiring that goes beyond a resume and a single video call, behavioral monitoring for the access-to-action speed that characterized the KnowBe4 incident, and, for open-source maintainers, requiring multiple independent reviewers for any change touching security-sensitive code regardless of a contributor's tenure.

9.2 Rug Pulls and Token Fraud (WA10): Cross-Layer Analysis

Rug pulls, introduced from the front-end and contract angle in Section 6.1.3, are worth revisiting here because the fraud rarely lives at a single layer: the social layer builds hype and community trust (Chapter 4 techniques), the front end presents a polished, professional-looking interface (Chapter 6), the contract contains the actual mechanism of extraction (Chapter 7), and the infra-

structure, a registered company, a funded marketing budget, sometimes even a fabricated audit report, lends borrowed legitimacy (this chapter). Treating a rug pull as purely a contract-layer bug misses most of the actual attack surface a defender or a diligence team should check.

Rug-pull red-flag checklist: - Liquidity not locked, or locked for a suspiciously short duration, in a verifiable on-chain lock contract. - Token contract unverified, or verified but containing an owner-gated transfer or sell restriction. - Anonymous team with no verifiable prior project history and no willingness to complete know-your-customer style diligence with reputable counterparties. - Marketing claims of an audit that, when checked, does not exist, is from an unverifiable auditor, or does not cover the deployed contract's actual bytecode. - Concentrated token holder distribution, with a small number of wallets, sometimes deployer-linked, holding a majority of supply.

9.3 Cloud, DNS, DDoS

Cloud misconfiguration remains one of the most common Web2/2.5 infrastructure failures feeding into Web3 incidents: publicly readable storage buckets containing deployment secrets, overly permissive Identity and Access Management (IAM) roles granting a compromised low-privilege credential a path to a high-privilege one, and leaked API keys committed accidentally to a public repository all recur across incident post-mortems industry-wide. DNS risk at the infrastructure level compounds registrar account security (Section 9.1.3) with the operational discipline of change management, every DNS record change should be logged, reviewed, and attributable to a specific authorized change request. Distributed denial-of-service (DDoS) attacks against exchange or RPC infrastructure serve a different objective than the theft-focused techniques elsewhere in this Part: they aim at availability, sometimes as pure extortion, sometimes as a diversion timed to draw incident-response attention away from a simultaneous theft attempt elsewhere in the stack.

Surface	Common failure	Detection	Primary control
Cloud storage	Publicly readable bucket with secrets	Automated public-exposure scanning	Default-private buckets, secret scanning in CI
IAM/cloud roles	Overly permissive role escalation path	Periodic access-graph review	Least privilege, scheduled access recertification
DNS change management	Unlogged/unattributed record change	Change-log reconciliation against records	Mandatory change tickets, RDAP monitoring
RPC/exchange availability	Volumetric or application-layer DDoS	Traffic-pattern anomaly detection	Rate limiting, upstream DDoS scrubbing, redundant providers

9.4 Insider and Third-Party Compromise

The Bybit incident's actual root cause, a compromised developer workstation at a third-party infrastructure vendor, illustrates a broader pattern worth naming explicitly: an organization's third-party vendors are part of its attack surface whether or not that organization treats them as

such. Vendor risk assessment should extend past a one-time security questionnaire at onboarding to continuous monitoring of any vendor with privileged access to production systems or signing infrastructure, and contractual requirements for prompt incident notification. Insider risk (Section 9.1.2) and third-party risk share a mitigation core, least-privilege access, separation of duties, and monitored, revocable credentials, because both describe a trusted party whose access is misused, whether through malice, coercion, or compromise of the party itself. The Employee Lifecycle Security Handbook (03) covers onboarding, role-change, and offboarding controls for internal personnel in full depth; treat every high-privilege vendor relationship under an equivalent lifecycle discipline rather than a lighter one.

9.5 Technique Catalog and TTP IDs

TTP ID	Technique	Top 15 ID	Primary detection signal	Primary mitigation
T9.001	Centralized exchange / Web2/2.5 infrastructure breach	WA07	Hot-wallet balance inconsistent with recorded withdrawals	Cold-storage majority custody, MPC/hardware signing
T9.002	Insider threat / collusive abuse	WA12	Access pattern outside behavioral baseline	Least privilege, separation of duties, access recertification
T9.003	DNS/ registrar/ routing hijack (infrastructure side)	WA13	Unattributed DNS or nameserver change	Registrar 2FA, registry lock, RPKI
T9.004	Nation-state fake hiring / malicious OSS contribution	WA15	Access-to-action speed after onboarding; identity inconsistencies	Enhanced hiring verification, multi-reviewer release gates
T9.005	Rug pull cross-layer orchestration	WA10	Unlocked liquidity, owner-gated token functions, unverifiable audit claims	Liquidity-lock verification, contract diligence checklist
T9.006	Cloud misconfiguration / credential leakage	(infra, non-WA)	Public-exposure scan hit; secret found in repository	Default-private storage, CI secret scanning
T9.007	DDoS extortion against exchange/ RPC infrastructure	(infra, non-WA)	Volumetric traffic anomaly, often timed with another incident	Rate limiting, DDoS scrubbing, redundant providers

Key controls for Part 2

- Treat the wallet signing dialog as the single highest-leverage control point in the stack: clear-signed, decoded summaries and independent transaction simulation stop multisig hijacking, approval abuse, and drainer kits at the same junction.
- Run every untrusted code path (candidate take-home projects, new dependencies, browser extensions) in isolation from any device or account that holds keys.
- Pin and verify the front-end delivery chain end to end: dependencies, build artifacts, CDN content, and DNS records, since an attacker only needs the one link a defender left unpinned.
- Extend contract-layer verification (SCVS, SCWE, and standard test coverage for reentrancy, oracle, access-control, and logic classes) rather than assuming the Top 15's non-contract focus means contract risk is out of scope.
- Validate RPC endpoints and chain identifiers before trusting displayed balances or broadcasting transactions, and cross-check against a second provider when a discrepancy appears.
- Manage insider, vendor, and nation-state hiring risk with the same access-lifecycle discipline applied to every other credential in the stack: least privilege, monitored access, and prompt revocation.

Part III: TTP Matrix and Use

Part I fixed the six-layer taxonomy and Part II filled it with a technique catalog, layer by layer. This part turns that catalog into a single working matrix: a tactic-technique-procedure (TTP) table with a fixed ID scheme, detection signals, and mitigations attached to every row, cross-walked to the OWASP Web3 Attack Vectors Top 15 (WA01 through WA15) and to MITRE's ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) and AADAPT (Adversarial Actions in Digital Asset Payment Technologies) frameworks. It then shows the matrix in use: as a threat-modeling input and as a detection and incident-response reference.

10. Tactic-Technique-Procedure Matrix

A TTP matrix is only useful if every row answers three questions the same way every time: why the adversary acted (tactic), how they did it (technique), and what a specific instance of that technique looked like in practice (procedure). This chapter fixes the structure so Parts II and IV can populate and extend it without drifting into inconsistent naming.

10.1 Structure and Naming Conventions

MITRE's own definition anchors the hierarchy: a tactic is the adversary's short-term technical goal, a technique is the means by which that goal is achieved, and a procedure is the specific implementation an adversary or campaign actually used (MITRE ATT&CK Design and Philosophy). This handbook does not invent a parallel tactic taxonomy. Every tactic ID in the matrix is a real ATT&CK tactic ID (`TA####`) or, where the action is native to value transfer rather than intrusion, a real AADAPT tactic ID (`ADTA####`), covered in full in Section 13.2. Reusing upstream tactic IDs keeps every row queryable against the source frameworks without a translation layer.

Techniques are where a generic enterprise framework runs out of vocabulary. "Approval phishing" and "multisig signing-UI manipulation" have no clean ATT&CK equivalent, so this handbook assigns its own technique IDs, scoped to the six layers from Part I, Section 2.1: `W3TTP-<LAYER>-<NNN>`, where `LAYER` is one of `USR` (User and Social), `WAL` (Wallet and Key Management), `DAP` (dApp and Front-End), `SCP` (Smart Contract and Protocol), `NOD` (Node and RPC), or `INF` (Infra-

structure), and `NNN` is a zero-padded, never-reused sequence number within that layer. A retired technique keeps its ID and is marked deprecated rather than recycled, matching ATT&CK's own numbering discipline. Procedures are not separately numbered; they are named, cited instances listed under a technique, exactly as ATT&CK lists "Procedure Examples" under each technique page.

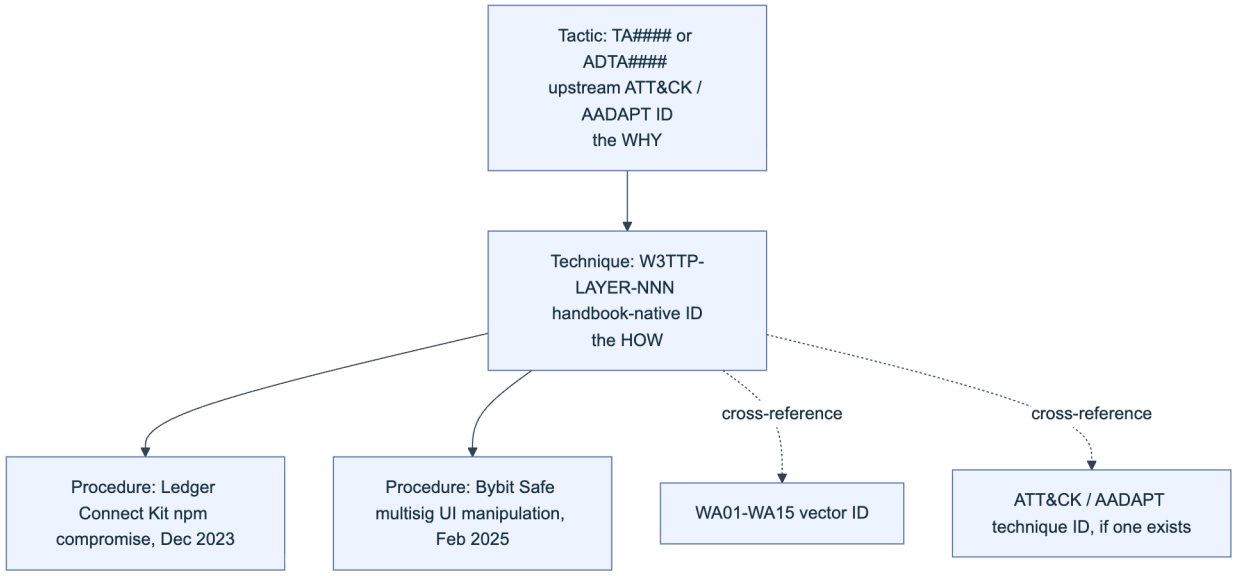

Figure 6. The three-level TTP hierarchy used throughout this handbook. Tactic IDs are always borrowed from upstream frameworks; technique IDs are handbook-native and layer-scoped; procedures are named examples, not separately numbered.

10.2 Detection and Mitigation Mapping

A technique row that names an attack without telling a defender what to watch for or what to build is a glossary entry, not a control. Every row in the working matrix carries two additional fields beyond the ID and the tactic: a **detection signal**, phrased as something a log source, monitoring rule, or on-chain watcher can actually emit, and a **primary mitigation**, phrased as a control a team can implement or a product can ship. Detection signals draw from two telemetry classes that this handbook treats as equally first-class: off-chain signals (endpoint detection and response, or EDR; DNS and certificate-transparency logs; SIEM correlation) and on-chain signals (event logs, mempool observation, state-diff simulation). The Incident Response Handbook (06) consumes this column directly as its triage reference, so the phrasing stays operational rather than descriptive: "detect X" rather than "X is dangerous."

The table below is a representative slice of the working matrix, one to three rows per layer, chosen to span the full outline of Part II. The complete matrix (all technique IDs, all procedure citations) lives in the Part V Appendix B quick-reference table; this slice illustrates the pattern every row follows.

TTP ID	Layer	Technique	Tactic (source ID)	WA ref.	Detection signal	Primary mitigation
W3TTP-USR-001	User	Fake interview / meeting-software lure	Initial Access (TA0001)	WA05	Unsigned installer fetched right after a video-call link; traffic to a new meeting-clone domain	Disposable VM for external calls; block unsigned binary execution
W3TTP-USR-002	User	Seed-phrase and credential phishing	Credential Access (TA0006)	WA08	Click-through on look-alike domains; DMARC/DKIM failure reports	Hardware wallet with on-device verification; phishing-resistant WebAuthn
W3TTP-USR-003	User	Pig-butchering / romance-investment grooming	Fraud (ADTA0001)	WA09	Large transfer to a newly funded counterparty after weeks of staged "profit" withdrawals	Velocity holds on first-time high-value counterparties
W3TTP-WAL-001	Wallet	Multisig signing-UI manipulation	Defense Evasion (TA0005)	WA01	Signer's independent calldata decode does not match the UI's rendered summary	Independent hardware verification of raw calldata before signing
W3TTP-WAL-002	Wallet	Unsecured private-key or seed exposure	Credential Access (T1552.004, *Private Keys*)	WA03	Key file access from an unexpected process; DLP hit on seed-phrase-shaped strings	Hardware security module or air-gapped signer; secret scanning
W3TTP-WAL-003	Wallet	Drainer contract / malicious approval	Fraud (ADTA0001) + Impact (TA0040)	WA04	`Approval` / `ApprovalForAll` to a newly deployed, unverified spender	Pre-sign approval simulation; periodic allowance revocation

TTP ID	Layer	Technique	Tactic (source ID)	WA ref.	Detection signal	Primary mitigation
W3TTP-DAP-001	dApp	Front-end JS supply-chain compromise	Initial Access (T1195.002)	WA02	SRI hash mismatch or CSP violation report; new script origin in production	SRI and CSP (Handbook 01); signed, pinned builds
W3TTP-DAP-002	dApp	Deceptive dApp interface / approval phishing	Defense Evasion (TA0005)	WA06	Rendered DOM diverges from the payload actually signed	Transaction simulation shown pre-sign; wallet-side domain risk scoring
W3TTP-DAP-003	dApp	DNS or domain hijack redirect	Resource Development (T1584.001, *Domains*)	WA13	Unexpected DNS record change; CT-log entry for an unrequested issuer	Registrar lock and DNSSEC (Handbook 02); CAA records
W3TTP-SCP-001	Contract	Oracle price manipulation	Impact (TA0040)	n/a (SCWE-ORACLE)	Spot price deviates from oracle beyond a set band after a large flash-loan trade	TWAP/multi-source oracles; circuit breakers on price-dependent state
W3TTP-NOD-001	Node/RPC	Eclipse attack / RPC manipulation	Collection (TA0009)	n/a	Peer-diversity drop; inconsistent state roots across independent RPC providers	Multi-provider RPC quorum; self-hosted fallback node
W3TTP-INF-001	Infrastructure	Insider-assisted access / collusion	Fraud (ADTA0001) + Valid Accounts (T1078)	WA12	Privileged action outside normal hours or geography; dual-control bypass	Least privilege with dual control on hot-wallet operations

10.3 Mapping Matrix Rows to WA01–WA15

The WA reference column above only works as a lookup if the mapping runs in both directions. An executive reading the OWASP Web3 Attack Vectors Top 15 as an awareness list needs to find which matrix rows, and therefore which controls, back a given WA vector. A defender who just

closed a matrix row after an incident needs to know which awareness-list item to update. This crosswalk table is that bridge, built directly from the layer assignments each vector carries in Part II's chapter structure: a vector that spans two layers (WA06 and WA13 each appear in two Part II chapters) gets a primary and secondary layer rather than a forced single answer.

WA ID	Vector	Primary layer(s)	Primary TTP ID(s)
WA01	Multisig Hijacking	Wallet	W3TTP-WAL-001
WA02	Supply Chain Attacks (npm, PyPI, OSS)	dApp	W3TTP-DAP-001
WA03	Private Key Compromise	Wallet	W3TTP-WAL-002
WA04	Drainer Malware and Drainer-as-a-Service	Wallet	W3TTP-WAL-003
WA05	Fake Interview and Video Call Social Engineering	User	W3TTP-USR-001
WA06	UI/UX Spoofing and Approval Phishing	Wallet, dApp	W3TTP-WAL-001, W3TTP-DAP-002
WA07	Centralised Exchange and Web2/2.5 Infrastructure Breaches	Infrastructure	W3TTP-INF-001
WA08	Phishing and General Social Engineering	User	W3TTP-USR-002
WA09	Romance, Investment, Impersonation, Recovery, and Pig Butchering Scams	User	W3TTP-USR-003
WA10	Rug Pulls, Fake Airdrops, and Token Impersonation	dApp	W3TTP-DAP-003
WA11	Wrench Attacks and Physical Coercion	User	W3TTP-USR-004
WA12	Insider Threats and Collusive Abuse	Infrastructure	W3TTP-INF-001
WA13	DNS, Domain, and Routing Infrastructure Hijacking	dApp, Infrastructure	W3TTP-DAP-003
WA14	Wallet Software, Extension, and App Compromises	Wallet	W3TTP-WAL-004
WA15	Nation-State Infiltration via Fake Hiring and Malicious OSS Contributions	Infrastructure	W3TTP-INF-001

11. Using the Matrix for Threat Modeling

The matrix earns its place in a threat-modeling session by replacing brainstorming with lookup: instead of asking “what could go wrong here,” a modeler asks “which matrix rows apply to this asset’s layer,” and works from a bounded, cited list.

Start from the scope, asset, and boundary work in Part I, Section 3.1: for each in-scope asset, identify its layer (a hot wallet is `WAL`, a swap front end is `DAP`, an RPC gateway is `NOD`), then pull every matrix row tagged to that layer plus every row on an adjacent layer the asset directly trusts. A custody desk evaluating a new multisig signer, for example, pulls all `WAL` rows and the `USR` rows for the humans who operate it, because W3TTP-USR-001 (fake-interview lure) is how an attacker gets malware onto the signer’s machine in the first place. From the pulled rows, build attack trees the same way Part I, Section 3.2 applies STRIDE: each row becomes a labeled path from precondition to impact, and the team scores likelihood against known deployment context (does this org use a browser extension wallet in scope for WA14, does it hold funds that make it a target for WA11) rather than against OWASP’s own explicit “not ranked” prevalence claim (covered in Section 13.1). The output is a per-asset table of applicable TTP IDs, current mitigation status, and an owner, which becomes the input to the SCSVS control-mapping exercise the session closes with.

Figure 7. The matrix-driven threat-modeling workflow. Every step resolves to a cited matrix row rather than an unstructured guess.

A worked fragment for a swap front end illustrates the output format: W3TTP-DAP-001 (front-end supply-chain compromise), mitigation status “SRI partially deployed, third-party analytics script unpinned,” owner “front-end lead,” residual risk “Medium until analytics script is pinned or removed.” That single line is simultaneously a threat-model entry, an SCSVS gap, and (Chapter 12) a detection-engineering backlog item, which is the point of keeping one matrix instead of three separate documents.

12. Using the Matrix for Detection and IR

A detection team reads the same matrix top to bottom for a different purpose: turning the Detection Signal column into alerting rules and turning the Primary Mitigation column into pre-authorized containment actions, so the first ten minutes of an incident are a lookup, not a debate.

Prioritize instrumentation by tactic stage. Early-stage tactics such as Reconnaissance (`TA0043`) and Resource Development (`TA0042`, which covers domain and infrastructure staging for WA13-class attacks) generate cheap, high-volume signals: a newly registered look-alike domain, a certificate-transparency entry, a GitHub fork of a wallet-connector repo. These are individually

low-confidence but valuable as an early-warning feed when correlated against brand and dependency inventories. Late-stage tactics such as Impact (`TA0040`) and AADAPT's Fraud (`ADTA0001`) generate expensive, high-confidence signals (a drained wallet, a manipulated oracle price), but by the time they fire the containment window has mostly closed. A mature program instruments both ends: cheap early signals to buy response time, and expensive late signals as the backstop. Every alert a SOC builds should carry the TTP ID that triggered it, so triage staff jump directly to that row's mitigation column and the matching playbook in the Incident Response Handbook (06) instead of reasoning from scratch under time pressure.

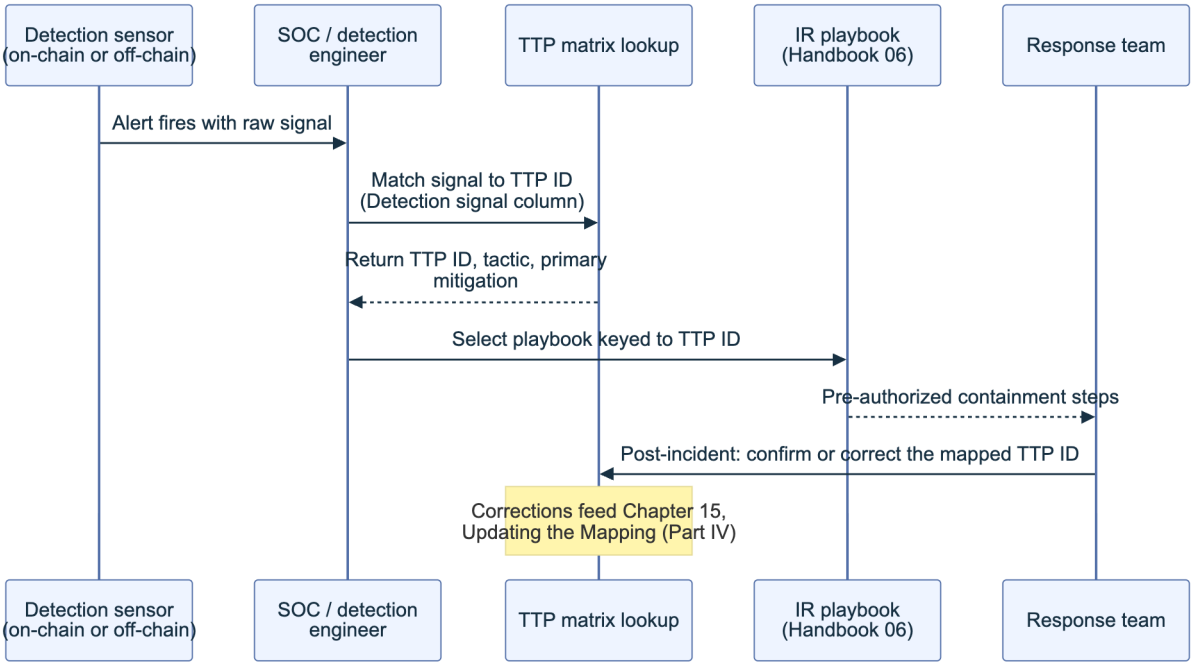

Figure 8. Matrix-driven detection and incident response. The TTP ID is the join key between the alert, the playbook, and the post-incident update.

A short triage checklist operationalizes the diagram: confirm the alert against the matrix row's stated detection signal before escalating (rules out signal drift); pull the mapped mitigation column as the first containment candidate rather than improvising; page the asset owner recorded during the Chapter 11 threat-modeling pass, since that person already has context on the asset's residual risk; and log whether the live incident matched the mapped TTP ID exactly or revealed a gap, feeding Part IV, Chapter 15's update process.

13. Alignment with OWASP Web3 Attack Vectors Top 15 and MITRE

The matrix is only as credible as the frameworks it cites. This chapter documents exactly what each upstream source claims for itself, so the crosswalk tables above are read with the right caveats attached rather than treated as more authoritative than their sources intend.

13.1 Top 15 (WA01–WA15) as Awareness and Checklist

The canonical OWASP Web3 Attack Vectors Top 15 is explicit about its own status: it complements the OWASP Smart Contract Top 10:2026 by covering non-smart-contract vectors, and the list is **not ranked** by loss volume, incident frequency, or severity. "Ordering does not imply relative severity or prevalence," the page states, and adds that ranking methodology, data sources, and update cycles are still being established. Treat WA01 through WA15 as a checklist of vectors a Web3 organization must be able to say something about, not as a priority-ordered backlog. This distinction matters operationally: a team that reads WA01 (listed first) as "the most important attack" and under-resources WA15 (listed last, nation-state infiltration) because of list position has misread the document.

Used correctly, the Top 15 is a coverage checklist run at the organization level, independent of any single asset's threat model. The table below is that checklist in template form: an organization fills the last two columns during a quarterly review, and a blank cell in "Control owner" is itself a finding.

WA ID	Vector	Assessed this cycle?	Control owner
WA01	Multisig Hijacking		
WA02	Supply Chain Attacks (npm, PyPI, OSS)		
WA03	Private Key Compromise		
WA04	Drainer Malware and Drainer-as-a-Service		
WA05	Fake Interview and Video Call Social Engineering		
WA06	UI/UX Spoofing and Approval Phishing		
WA07	Centralised Exchange and Web2/2.5 Infrastructure Breaches		
WA08	Phishing and General Social Engineering		
WA09	Romance, Investment, Impersonation, Recovery, and Pig Butchering Scams		
WA10	Rug Pulls, Fake Airdrops, and Token Impersonation		
WA11	Wrench Attacks and Physical Coercion		
WA12	Insider Threats and Collusive Abuse		

WA ID	Vector	Assessed this cycle?	Control owner
WA13	DNS, Domain, and Routing Infrastructure Hijacking		
WA14	Wallet Software, Extension, and App Compromises		
WA15	Nation-State Infiltration via Fake Hiring and Malicious OSS Contributions		

13.2 Cross-Mapping to MITRE ATT&CK and AADAPT

The Top 15 names vectors; it does not define reusable tactic and technique identifiers, procedure catalogs, or a versioned data model. MITRE's ATT&CK and AADAPT frameworks supply exactly that, and the matrix in Chapter 10 borrows their IDs rather than duplicating their content. The two frameworks divide the space differently and the split matters for how you cite them: ATT&CK covers the intrusion lifecycle common to any networked system, while AADAPT covers the actions specific to moving, minting, burning, and laundering value on a ledger, actions that have no equivalent in a conventional enterprise kill chain.

13.2.1 MITRE ATT&CK (Enterprise, etc.)

MITRE ATT&CK publishes three matrix domains: Enterprise (IT networks and cloud), Mobile (device-resident threats), and ICS (industrial control systems). This handbook draws almost entirely from Enterprise, whose current tactic set runs 14 tactics from Reconnaissance (TA0043) through Impact (TA0040), and occasionally from Mobile for wallet-app-specific vectors under WA14, since a mobile wallet app compromise fits Mobile's device-permission model better than Enterprise's network model. Each ATT&CK technique carries a stable `T####` ID, and where a technique has meaningfully distinct sub-variants, a dotted sub-technique ID such as `T1566.002` (Spearphishing Link, a sub-technique of T1566 Phishing). T1195 Supply Chain Compromise and its sub-technique T1195.002 Compromise Software Supply Chain are the technique pair behind every WA02 and WA13 matrix row in Chapter 10; T1584 Compromise Infrastructure and its Domains and Web Services sub-techniques back the DNS and hosting rows.

Figure 1. The ATT&CK for Enterprise Matrix

Figure. ATT&CK's Enterprise matrix is a behavioral coverage map, not a chronological checklist: a campaign may enter at several tactics, repeat techniques, or skip columns. Use it to identify detection and control gaps around Web3 infrastructure, then use AADAPT for ledger-native behaviors the Enterprise matrix does not name. Source: MITRE ATT&CK Design and Philosophy, © MITRE; approved for public release and unlimited distribution.

Web3 TTP ID	ATT&CK ID	ATT&CK name	Why it applies
W3TTP-USR-002	T1566	Phishing	Seed-phrase and credential harvesting through crafted messages
W3TTP-DAP-001	T1195.002	Compromise Software Supply Chain	JS package or CDN artifact tampering before it reaches a browser
W3TTP-DAP-003	T1584.001	Domains	Staging or hijacking a domain used for a phishing redirect
W3TTP-WAL-002	T1552.004	Unsecured Credentials: Private Keys	Seed phrase or key material found in plaintext storage
W3TTP-INF-001	T1078	Valid Accounts	Insider or ex-employee credential reused for unauthorized privileged access

13.2.2 MITRE AADAPT Tactics and Techniques for Digital Assets

AADAPT (Adversarial Actions in Digital Asset Payment Technologies) is MITRE's ATT&CK-style knowledge base scoped to digital-asset systems, published openly on GitHub and described on the project site as complementary to ATT&CK, not a replacement for it. The version examined here (`AADAPT.yaml`, version 4.4.0, retrieved 10 August 2026) defines 11 tactics: 10 reuse ATT&CK Enterprise tactic IDs verbatim (Reconnaissance, Resource Development, Initial Access, Execution, Privilege Escalation, Defense Evasion, Credential Access, Lateral Movement, Collection, Impact), and one, `ADTA0001` **Fraud**, is new: "the adversary is trying to illicitly create, acquire, or utilize value-form," a goal ATT&CK's intrusion-centric model has no category for. Underneath sit 66 techniques, most native to blockchain mechanics: Oracle Manipulation (`ADT3012.004`), Reentrancy (`ADT3012.005`), Flash Loan (`ADT3015`), Signature Replay Attack (`ADT3012.006`), Chain Reorganization (`ADT3003`), and Eclipse Attack (`ADT3006`), alongside laundering typologies such as Money Mules (`ADT3028.004`), Peel Chains, CoinJoin, and Tumblers that give the `INF` and `WAL` layers a shared post-theft vocabulary.

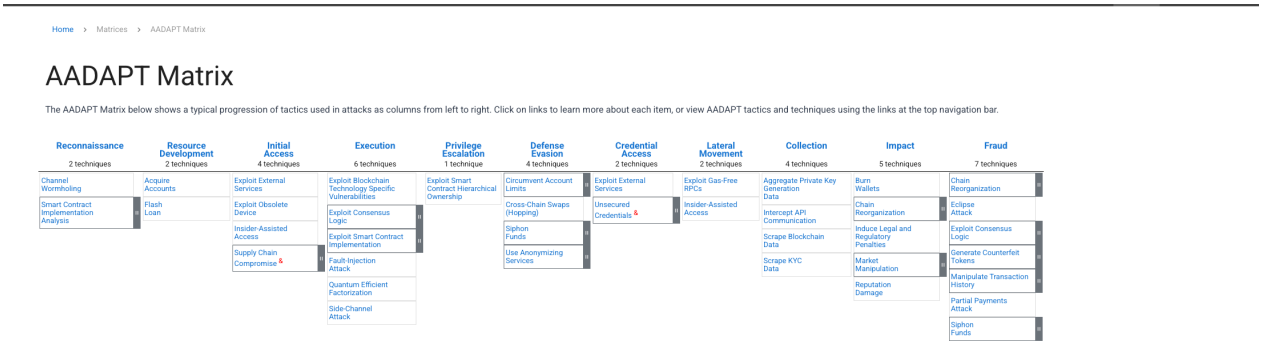

Figure. The AADAPT matrix extends ATT&CK's behavior-first structure into digital-asset payment technology, making contract exploitation, consensus abuse, key compromise, fund movement, and laundering visible in one coverage map. Source: MITRE AADAPT Matrix, © 2025 MITRE, approved for public release and unlimited distribution (case 25-1204); reproduced for research and education under the AADAPT terms.

One entry is worth flagging because it shows the two frameworks converging rather than staying permanently parallel: AADAPT's `ADT1552.004` **Private Keys** technique carries an explicit `ATT&CK-reference` pointing at T1552/004, meaning it was contributed upstream and now exists as a registered ATT&CK Enterprise sub-technique, not merely a cross-reference. Most other AADAPT techniques remain AADAPT-native (`ADT####` with no upstream mirror), so check both catalogs rather than assuming AADAPT is a strict subset.

AADAPT ID	Name	Tactic	Web3 TTP ID	WA ref.
ADT3012.004	Oracle Manipulation	Impact (TA0040)	W3TTP-SCP-001	n/a (SCWE-ORACLE)
ADT3015	Flash Loan	Impact (TA0040)	W3TTP-SCP-001	n/a (SCWE-ORACLE)
ADT3006	Eclipse Attack	Collection (TA0009)	W3TTP-NOD-001	n/a

AADAPT ID	Name	Tactic	Web3 TTP ID	WA ref.
ADT3020.001	Address Poisoning	Fraud (ADTA0001)	W3TTP-DAP-002	WA06
ADT3028.004	Money Mules	Fraud (ADTA0001)	W3TTP-INF-001	WA12
ADT1552.004	Private Keys	Credential Access (TA0006 / T1552.004)	W3TTP-WAL-002	WA03

13.3 Web3-Specific Extensions and Community Mappings

Neither ATT&CK nor AADAPT was written to describe a spoofed wallet-connect button or a cloned airdrop-claim page in detail, because both frameworks work at the level of intrusion and value-transfer mechanics, not front-end deception. That gap is why a community layer of Web3-specific mappings exists alongside the two MITRE frameworks, and why this handbook treats them as first-class sources rather than footnotes.

The OWASP Web3 Attack Vectors Top 15 itself is the primary community mapping this handbook builds on, and its methodology is openly still forming, per Section 13.1's caveat. Security Alliance (SEAL), a non-profit crypto-native security coordination body, runs SEAL 911, a free around-the-clock hotline connecting anyone facing an active incident to a vetted responder pool, and a companion information-sharing network under the SEAL-ISAC banner; both feed real procedure examples into community catalogs faster than any single vendor's incident log could. DeFiHack-Labs maintains a large, growing public repository of DeFi incidents reproduced as executable Foundry proof-of-concept tests, giving the `SCP` layer a runnable procedure source rather than prose description alone. For the on-chain leaf specifically, the Smart Contract Security Weakness Enumeration (SCWE) supplies 156 weakness entries across 11 categories aligned to SCSVS sections, which is what the `n/a (SCWE-ORACLE)` references above point back to.

Community-sourced technique candidates do not enter the matrix automatically. Part IV, Chapter 15 defines the review and versioning process that promotes a candidate from "observed in the wild, documented by SEAL or DeFiHackLabs" to "assigned a `W3TTP-<LAYER>-<NNN>` ID with a detection and mitigation pair," keeping the matrix a curated reference rather than an unfiltered aggregation feed.

Key controls for Part 3

- Every matrix row carries a tactic ID borrowed from ATT&CK or AADAPT, a handbook-native `W3TTP-<LAYER>-<NNN>` technique ID, at least one cited procedure, a detection signal, and a primary mitigation.
- Keep the WA01-WA15 crosswalk bidirectional: every matrix row maps to a WA vector where one exists, and every WA vector resolves to at least one matrix row.

- Cite ATT&CK and AADAPT IDs directly rather than re-deriving technique descriptions; check both catalogs, since only a minority of AADAPT techniques (for example `ADT1552.004`) are formally merged into ATT&CK.
- Treat the OWASP Web3 Attack Vectors Top 15 as an unranked coverage checklist, never as a priority-ordered backlog.
- Route detection-signal and mitigation columns into the Incident Response Handbook (06) playbooks so triage resolves to a lookup, not a fresh investigation.
- Feed post-incident TTP corrections and community-sourced technique candidates through the Part IV, Chapter 15 update process rather than editing the matrix ad hoc.

Part IV: Case Studies and Examples

Every control catalogued in Parts II and III exists because a real attacker used a real technique against a real system. This part tests the six-layer mapping against five incidents that together touch every layer in the taxonomy, from a $1.5 billion multisig and infrastructure compromise down to a single trojanized browser-extension update. It closes by treating the mapping itself as a maintained artifact: how the community proposes new techniques, and how this handbook stays synchronized with the OWASP Web3 Attack Vectors Top 15, the Smart Contract (SC) Top 10, and the Smart Contract Weakness Enumeration (SCWE) as each evolves on its own schedule.

14. End-to-End Attack Scenarios

A model that treats an incident as either “a contract bug” or “a user mistake” undercounts how these attacks actually unfold. The five case studies below each start in one layer, pivot through one or two more, and land their impact in a different layer again, which is the pattern a threat model needs to anticipate rather than the exception it needs to excuse.

14.1 Multi-Layer Scenarios (Including Top 15 Case Studies)

An attacker rarely needs to break the layer that holds the money. It is usually cheaper to break a layer that the money-holding layer implicitly trusts, then let that trust carry the attack the rest of the way. A cold-storage multisig trusts the software that renders its transactions. A dApp trusts the npm package it imported for wallet connection. A browser extension trusts its own auto-update channel. A hiring pipeline trusts a resume and a video call. In every case in this chapter, the layer that was actually broken is not the layer where the loss was recorded.

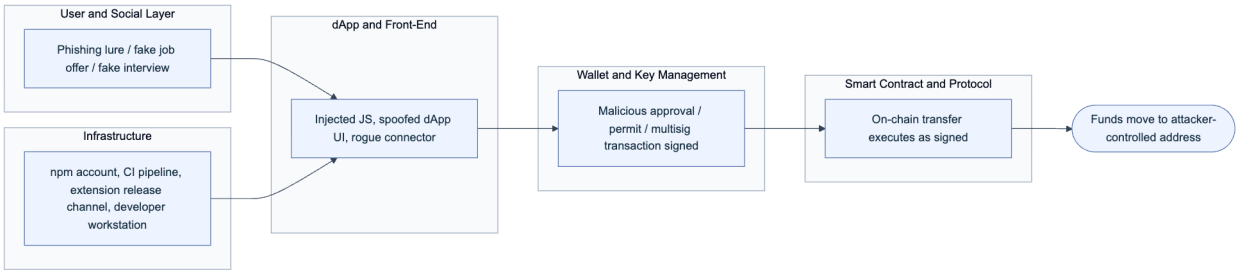

Figure 6. The recurring multi-layer pattern: the layer an attacker compromises (User/Social or Infrastructure) is rarely the layer where the theft is recorded (Wallet and Smart Contract). Node and RPC involvement varies by incident and is omitted here for clarity.

The table below previews how each case study in Section 14.2.1 maps onto this pattern, naming the layer where the attacker actually gained a foothold versus the layer where the loss became visible, and the OWASP Web3 Attack Vectors Top 15 (WA) identifiers each incident is filed under.

Incident	Foothold layer	Pivot layer	Loss recorded at	WA ID(s)
Bybit	Infrastructure (Safe developer workstation)	dApp and Front-End (injected JS in signing UI)	Wallet and Key Management (multisig execution)	WA01, WA07
Ledger Connect Kit	Infrastructure (phished npm account)	dApp and Front-End (poisoned connector library)	Wallet and Key Management (rerouted approval)	WA02
Drainers / approval phishing	User and Social Layer (phishing lure)	dApp and Front-End (spoofed claim/mint site)	Wallet and Key Management (signed approval or permit)	WA04, WA06
Trust Wallet extension	Infrastructure (extension release pipeline)	Wallet and Key Management (the wallet software itself)	User and Social Layer (seed phrase exfiltrated)	WA14
DPRK TraderTraitor	User and Social Layer (fake hiring, fake interview)	Infrastructure (insider or CI access)	Varies by operation; feeds forward into incidents like Bybit	WA15

Reading down the “foothold” and “loss recorded at” columns side by side is the exercise this handbook exists to support: a defense concentrated only on the layer where money sits will miss every one of these five attacks.

14.2 Reference to Public Incidents and RCAs

A mapping that is not anchored to public root cause analyses (RCAs) drifts into abstraction and stops being testable. MITRE ATT&CK earns its authority the same way: every technique traces to observed intrusions, not hypothesized ones (MITRE ATT&CK). The five incidents below were chosen because each has a citable, technically detailed public account from the affected party, a blockchain intelligence firm, or a federal agency, and because together they exercise five different

WA identifiers without overlapping mechanism. Read each case study as a worked example of Section 11's threat-modeling method: identify the layer that actually failed, then check whether your own environment has the equivalent control gap.

14.2.1 Five named incidents across the Top 15

Bybit: a $1.5 billion multisig and infrastructure compromise (WA01, WA07)

On February 21, 2025, the cryptocurrency exchange Bybit lost approximately 401,000 ETH, worth close to $1.5 billion at the time, in what blockchain intelligence firm Chainalysis called the largest digital asset theft on record (Chainalysis). The theft did not exploit Bybit's own systems. The investigation traced the root cause to a compromised developer workstation at Safe{Wallet}, the third-party multisig infrastructure Bybit used to manage its cold wallet (Sygnia). The attacker used that access to insert malicious JavaScript into the Safe front end, so that when Bybit's approvers reviewed what looked like a routine cold-to-hot wallet transfer, the interface displayed a benign summary while the transaction actually being signed reassigned control of the wallet. Because Safe's security model assumes signers review what they sign, corrupting that one shared surface defeated the multisig threshold without touching a single private key. This is a compound of WA01 (Multisig Hijacking) at the point of execution and WA07 (Centralised Exchange & Web2/2.5 Infrastructure Breaches) at the point of entry, since Safe's web application is Web2.5 infrastructure the exchange depended on without directly controlling.

Field	Detail
Date	February 21, 2025
Loss	~401,000 ETH, ~$1.5B
Attribution	Lazarus Group / TraderTraitor (FBI)
Entry point	Compromised Safe developer workstation
Mechanism	Malicious JS altered signed transaction while UI showed a normal transfer

The FBI's public service announcement of February 26, 2025 formally attributed the theft to North Korea's "TraderTraitor" actors and published 51 Ethereum addresses tied to the stolen funds for industry-wide blocking (FBI IC3 PSA 250226). Bybit replenished its reserves within 72 hours through bridge financing from Galaxy Digital, FalconX, and Wintermute, so no customer suffered a loss, but the incident remains the reference case for why a signing interface is itself part of the attack surface.

Detection signals: raw calldata that does not match the UI-rendered transaction summary; unscheduled or off-hours multisig approval requests; unexpected delegatecall or ownership-changing targets in a routine transfer; integrity-hash drift on the signing front end's served JavaScript.

Mitigations: verify transaction calldata on an independent, air-gapped device rather than trusting the browser-rendered summary; run an independent transaction simulator (a second implementation, not the same vendor's UI) before any signer approves a high-value transfer; apply the front-end supply chain controls from the CDN and Front-End Supply Chain Security Handbook (01), including Subresource Integrity (SRI) on the signing application itself, to any third-party multisig UI; segregate developer workstations with production deploy access from general-purpose browsing and email.

Ledger Connect Kit: a poisoned wallet-connector library (WA02)

On December 14, 2023, an attacker who had phished a former Ledger employee's npm account published three malicious versions (1.1.5 through 1.1.7) of `@ledgerhq/connect-kit`, a JavaScript library dozens of decentralized applications (dApps) embed to let users connect a wallet (Ledger). The tampered package substituted a rogue WalletConnect project that silently rerouted approved transactions to an attacker-controlled address. Ledger's own hardware and Ledger Live were untouched; every affected user was a customer of some other dApp that had added Connect Kit as a dependency, often years earlier, and never re-verified it. The malicious code was live for roughly five hours, with an actual drain window under two hours before a fix shipped, and The Block reported roughly $600,000 drained across affected dApps before the package was pulled from the registry.

This is the code-level supply chain failure mode covered in depth by the CDN and Front-End Supply Chain Security Handbook (01): a single compromised publisher credential propagated instantly to every project that resolved the package's version range, with no re-verification step in between. It is WA02 (Supply Chain Attacks) in its purest form, because the vulnerability was never in any dApp's own code.

Detection signals: an npm package publishing an unexpected new version outside its normal release cadence; a wallet-connection library requesting a different WalletConnect project ID than the one a dApp registered; sudden anomalous outbound approval destinations across multiple, otherwise unrelated dApps within the same short window.

Mitigations: pin exact dependency versions and hashes rather than floating ranges; apply Subresource Integrity to any CDN-served bundle of a wallet connector; require hardware-backed multi-factor authentication and short-lived, scoped tokens for every account with publish rights to a package used this widely; monitor Sigstore's Rekor transparency log or an equivalent for unexpected signing events tied to your own dependencies (Sigstore).

Drainers and approval phishing at scale (WA04, WA06)

Drainer-as-a-service (DaaS) turned wallet theft into a rentable product. A kit such as Inferno Drainer, which Group-IB tracked across more than 16,000 phishing domains impersonating over 100 legitimate brands, gave low-skill affiliates a ready-made phishing site that spoofed the connection flow of popular protocols including Seaport, WalletConnect, and Coinbase Wallet, then split the proceeds roughly 80/20 in the affiliate's favor (Group-IB). Group-IB estimated at least $80 million stolen through the kit before its operators announced a shutdown in November 2023, though the panel remained reachable months later, a reminder that a brand takedown does not remove the underlying infrastructure.

The technical hook in nearly every drainer campaign is the same regardless of kit: get a victim to sign an ERC-20 `approve` or EIP-2612 `permit` call, or an ERC-721/1155 `setApprovalForAll`, that grants an operator contract unlimited or broad transfer rights, then sweep the wallet on the drainer operator's own schedule rather than at signing time (Chainalysis). Because the malicious grant and the theft are separated in time, a victim frequently has no reason to suspect the wallet they connected weeks ago to claim a fake airdrop is the one now being drained. Industry tracker Scam Sniffer reported that aggregate phishing losses fell 83%, from roughly $494 million in 2024 to about $84 million in 2025, crediting wider adoption of wallet-side transaction simulation and community blocklists for the decline, even as the underlying kits remained active (Scam Sniffer).

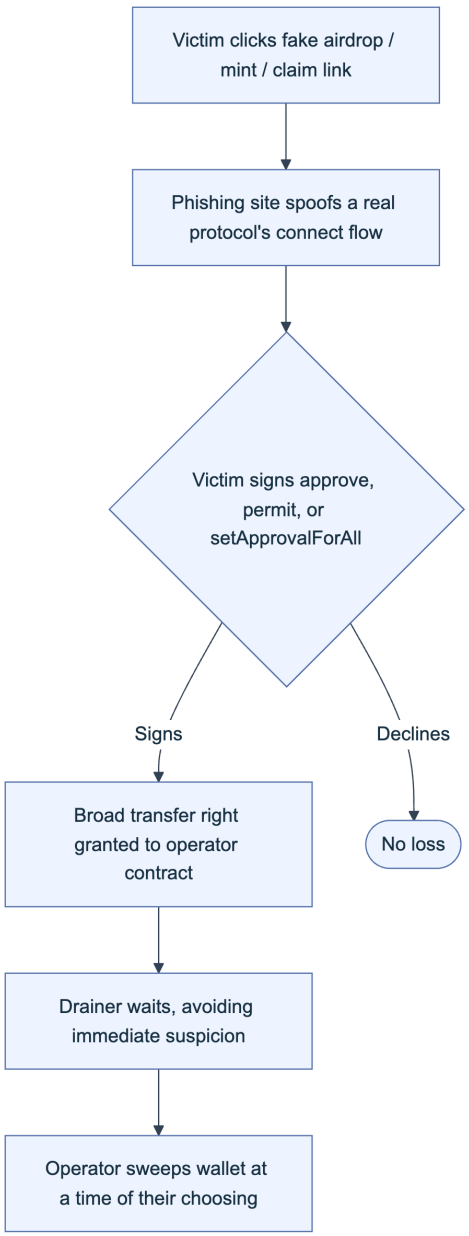

Figure 7. The approval-phishing kill chain. The delay between grant (P) and sweep (D) is what makes this pattern harder to notice than a same-block theft.

Detection signals: a wallet transaction requesting `approve` with `type(uint256).max` or `setApprovalForAll(operator, true)` to a contract with no prior reputation; a connection request whose domain does not match the protocol it visually impersonates; a sudden multi-asset outbound transfer with no preceding user-initiated action.

Mitigations: use wallets with transaction simulation that flags unlimited approvals before signing; grant time-boxed or amount-limited allowances instead of unlimited ones where the interface supports it; periodically audit and revoke stale approvals with a tool such as the

Etherscan Token Approval Checker or Revoke.cash; block known drainer infrastructure using community-maintained domain and contract blocklists; treat any unsolicited “claim your airdrop” prompt as a WA06 (UI/UX Spoofing & Approval Phishing) scenario by default.

Trust Wallet browser extension: a compromised official update (WA14)

On December 24, 2025, a compromised build of Trust Wallet’s Chrome extension, version 2.68.0, was distributed through the official Chrome Web Store, so users who installed or auto-updated received it as a routine, trusted release. Security researchers found the malicious behavior hidden inside a bundled file, `4482.js`, disguised as analytics code; it triggered when a user imported a seed phrase and exfiltrated wallet data to an external domain, `metrics-trustwallet[.]com` (BleepingComputer). Binance founder Changpeng Zhao confirmed roughly $7 million in affected funds and committed to covering the loss. Trust Wallet shipped a patched version, 2.69, the following day and urged every user on 2.68.0 to update immediately.

This incident is the wallet-software mirror of the Ledger Connect Kit case: rather than a dApp trusting a poisoned dependency, here the wallet itself was the poisoned artifact, delivered through the one channel (the official extension store) users have no practical way to route around. A second wave of harm followed the first: opportunistic phishing sites, including one at `fix-trustwallet[.]com`, capitalized on user panic by directly soliciting seed phrases under the guise of a fix, layering WA08 (Phishing & General Social Engineering) on top of the original WA14 event.

Detection signals: an extension auto-updating to a version number not announced in the vendor’s own release notes; new, previously unseen outbound network destinations appearing immediately after an extension update; a spike in seed-phrase-import events correlated with a specific extension version in telemetry.

Mitigations: publish and independently verify reproducible build hashes for every extension release so a compromised build artifact is externally detectable before mass distribution; minimize the extension’s requested host permissions so it cannot reach arbitrary domains even if compromised; keep high-value holdings on a hardware wallet whose seed material never touches software that receives remote updates; monitor vendor security advisories and be prepared to force-disable a specific extension version fleet-wide when operating a managed browser environment.

DPRK TraderTraitor: nation-state access through fake hiring (WA15)

TraderTraitor is the name the US Cybersecurity and Infrastructure Security Agency (CISA), the FBI, and the Treasury Department gave in a April 2022 joint advisory to a persistent North Korean campaign that distributes trojanized cryptocurrency trading and exchange applications, built by fake companies with polished websites and social media presence, to gain initial access into

blockchain and fintech firms (CISA AA22-108A). The same actor cluster is the one the FBI's 2025 advisory named in connection with the Bybit theft covered above, which is why this case belongs in the mapping as its own entry: Bybit describes what was broken, TraderTraitor describes the actor and access pattern that keeps recurring across many different victims.

A parallel, increasingly prominent branch of the same threat is the DPRK IT worker fraud scheme: thousands of North Korean nationals, using stolen or fabricated Western identities and sometimes facilitated by "laptop farms" run by co-conspirators inside the target country, obtain legitimate remote developer roles at crypto and technology companies. Salary is funneled back to the regime, and the resulting insider access has in multiple documented cases been leveraged for further compromise or extortion after termination. The US Department of Justice has brought multiple indictments against facilitators of this scheme since 2024. A related pattern targets developers directly: fake job interviews or coding-test repositories that, once cloned and run, execute malware on the candidate's own machine, a technique tracked under names including Contagious Interview.

Detection signals: inconsistent video-interview behavior or documentation for a remote hire; salary payments requested to money-mule or frequently-changed accounts; unsolicited take-home coding exercises that require running an unreviewed `npm install` or arbitrary setup script; new open-source contributor accounts adding obfuscated or unexplained dependencies; command-and-control (C2)-pattern outbound traffic from a developer laptop or a CI runner shortly after onboarding.

Mitigations: strengthen identity verification for remote technical hires beyond a resume and a single video call; sandbox and network-isolate any take-home interview exercise before execution; apply dependency review and software bill of materials (SBOM) checks to contributions from new or unverified accounts; revoke access and credentials immediately and completely at offboarding; cross-reference outbound crypto addresses against Treasury's Office of Foreign Assets Control (OFAC) sanctioned-address lists. This is precisely where the Employee Lifecycle Security Handbook (03) and the Hiring, Remote Work, and Insider Threat Handbook (05) take over from this one: those handbooks carry the HR-side controls that TraderTraitor's fake-hiring branch is designed to defeat.

---

15. Updating the Mapping

A TTP catalog that stops changing after publication stops describing the threat within a year. This chapter sets out how the mapping keeps pace: who can propose a change, how a version is cut, and how this handbook stays aligned with the three upstream OWASP artifacts it depends on.

15.1 Community Contribution and Versioning

The OWASP Smart Contract Security (SCS) project already runs a working contribution pipeline for its Top 10 and testing guide repositories: an idea starts as a GitHub Discussion, promising ideas are promoted to a formal issue, and a pull request (PR) against approved content is reviewed by the core team before merging, all published under a CC-BY-4.0 license (OWASP SCS contributing). This handbook's TTP mapping should use that same channel rather than a separate process.

For versioning cadence, MITRE ATT&CK is the closest working precedent at this handbook's scale: it ships on a semi-annual schedule, alternating roughly between April and October, using a major.minor scheme where a major version increment adds or restructures techniques and a minor increment carries corrections that do not change technique scope (ATT&CK reached v19.2 in April 2026 under this pattern) (MITRE ATT&CK versions). This handbook adopts the same discipline for its own layer catalog and TTP matrix: a major revision when a layer gains, loses, or restructures a technique family or when an upstream WA identifier is renumbered; a minor revision for new case studies, citation refreshes, or wording fixes that leave the taxonomy unchanged.

A contribution proposing a new technique or case study should arrive as a PR containing, at minimum, the fields below, matching the finding structure used throughout Part II so a reviewer can merge it without reformatting:

```
## Proposed addition

- **Layer**: [User/Social | Wallet | dApp/Front-End | Smart Contract/Protocol | Node/
      RPC | Infrastructure]
- **WA ID(s)**: [e.g. WA04, WA06]
- **Technique summary**: [1-3 sentences, methodology not just outcome]
- **Public evidence**: [link to an RCA, advisory, or post-mortem; no evidence, no
      merge]
- **Detection signal(s)**: [concrete, observable]
- **Mitigation(s)**: [concrete, actionable]
```

Reject any submission that lacks a cited, public source. This is the same anti-anecdote discipline MITRE ATT&CK enforces to keep the framework technique-grounded rather than speculative, and it is what keeps this handbook's case studies auditable rather than aspirational.

15.2 Integration with Web3 Attack Vectors Top 15, SC Top 10, and SCWE Updates

This handbook does not own the three catalogs it references; it tracks them. The Web3 Attack Vectors Top 15 supplies the WA identifiers used throughout Part IV. The SC Top 10 catalogs on-chain logic risk and is itself forward-looking: the 2026 edition already draws on 2025 incident data, citing roughly $905.4 million in losses across 122 deduplicated protocols to forecast the coming year's most significant risks (OWASP SC Top 10). The SCWE enumerates specific on-chain

weaknesses across eleven SCSVS-aligned domains, numbered SCWE-001 upward, explicitly positioned as a blockchain-focused complement to the Common Weakness Enumeration (CWE) rather than a replacement for it (OWASP SCWE). Because these three artifacts version independently of each other and of this handbook, a change in any one can silently invalidate a cross-reference in Chapter 7's contract-layer mapping or in the WA-ID columns used across this part.

The table below defines the trigger that should prompt a review of this handbook when each upstream artifact moves, and which chapter owns the resulting update.

Upstream artifact	Typical cadence	Update trigger for this handbook	Owning chapter
Web3 Attack Vectors Top 15	Irregular, OWASP SCS release cycle	Any WA ID added, renumbered, or retired	Chapters 2, 13, and every WA reference in Part IV
SC Top 10	Annual (forward-looking edition)	New or reordered SC entries affecting contract-layer techniques	Chapter 7 (Part II)
SCWE	Rolling, tied to SCSVS domains	New SCWE-### entries within an SCSVS domain this handbook already maps	Appendix D (Part V)
MITRE ATT&CK / AADAPT	Semi-annual (ATT&CK); rolling (AADAPT)	New tactic or technique with a plausible Web3 analogue not yet represented	Part II technique catalog

Treat this table as a recurring calendar item, not a one-time reconciliation. A practical cadence is to check the SC Top 10 and Web3 Attack Vectors Top 15 for changes each time either publishes, and to run a lighter SCWE and AADAPT diff on the same semi-annual rhythm this handbook uses for its own version bumps. Because MITRE AADAPT (Adversary Actions in Digital Asset Payment Technologies) extends the ATT&CK methodology specifically to digital-asset and blockchain payment threats, a new AADAPT technique is often the first external signal that a Web3-specific attack pattern deserves its own entry in Part II before it shows up as a named incident in this part (MITRE AADAPT).

Key controls for Part 4

- Verify what a signing interface displays against the raw calldata it actually submits, on hardware or an independent simulator, before approving any high-value multisig transaction.
- Pin dependency and extension versions with hashes, and monitor a transparency log such as Rekor for unexpected signing events on packages you depend on.
- Default to time-boxed or limited-value token approvals, and audit and revoke standing approvals on a recurring schedule.
- Verify reproducible build hashes for wallet software and browser extensions before trusting an auto-update.

- Treat unsolicited remote-hire pipelines and take-home coding exercises as an attack surface, and sandbox anything a candidate asks you to run.
- Review this handbook's WA-ID and SCWE cross-references every time the Web3 Attack Vectors Top 15, SC Top 10, or SCWE publishes a new version.

Part V: Appendices

This part collects the reference material the rest of the handbook assumes you already have to hand. Appendix A fixes the vocabulary so a term like *drainer-as-a-service* means the same thing in Chapter 5 as it does in Chapter 14. Appendix B condenses the six layer chapters of Part II into one scannable table. Appendices C and D turn the two external references this handbook is built on, the OWASP Web3 Attack Vectors Top 15 and the Smart Contract Weakness Enumeration (SCWE), into quick lookups, and Appendix E points to the full bibliography.

16. Appendix A: Glossary

Entries group by the part of the stack where the term does the most work, not strict alphabetical order: a reader triaging a wallet-drain report and a reader mapping a contract finding to SCWE reach for different clusters of vocabulary. Within each group, entries run alphabetically, acronyms expand on first appearance, and each entry names the chapter with the fuller treatment.

16.1 Frameworks, Standards, and Modeling Terms

These terms describe the reference frameworks this handbook maps every technique onto, introduced in Chapter 1.

- **MITRE AADAPT (Adversarial Actions in Digital Asset Payment Technologies)**: a MITRE knowledge base, launched in 2025 and built to complement ATT&CK, cataloging adversary tactics specific to digital asset and payment systems. See the MITRE fact sheet, the public repository, and Chapter 1.3.
- **MITRE ATT&CK**: a globally accessible knowledge base of adversary tactics and techniques drawn from real-world observations, structured as tactics, techniques, and sub-techniques across Enterprise, Mobile, and ICS domains. See attack.mitre.org and Chapter 1.2.
- **SC Top 10 (Smart Contract Top 10)**: OWASP's ranked list of the ten most critical on-chain vulnerability classes, currently the 2026 edition (SC01 through SC10). See scs.owasp.org/sctop10 and Chapter 1.4.

- **SCSVS** and **SCSTG**: the Smart Contract Security Verification Standard and its companion Testing Guide; this handbook extends their coverage to wallet, front-end, and infrastructure layers neither reaches. See scs.owasp.org and Chapter 1.4.
- **SCWE (Smart Contract Weakness Enumeration)**: OWASP's catalog of on-chain weakness classes, past 150 entries across eleven SCSVS domains, mapped in Appendix D. See scs.owasp.org/SCWE and Chapter 1.4.
- **STRIDE**: a threat-modeling mnemonic (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege) introduced by Loren Kohnfelder and Praerit Garg at Microsoft in 1999. See Microsoft's documentation and Chapter 3.2.
- **TTP (tactic, technique, and procedure)**: the tiered ATT&CK vocabulary this handbook borrows: tactic is the adversary's objective, technique the general method, procedure the specific observed case. See Chapter 1.2.
- **Web3 Attack Vectors Top 15**: OWASP's awareness list of the fifteen most significant non-smart-contract Web3 risks (WA01 through WA15), an alternate companion to the SC Top 10. See scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15 and Appendix C.

16.2 User, Wallet, and Social Engineering Terms

These terms describe the path from a human decision to a signed, irreversible transaction, the surface covered in Chapters 4 and 5.

- **Approval phishing**: tricking a user into granting a token `approve` or `setApprovalForAll` allowance to an attacker address, letting the attacker sweep the balance later without a further signature. Central to WA06.
- **Blind signing**: approving a transaction on a wallet that cannot decode and display its actual effect, forcing trust in the requesting front end's claim of what it does. See the UX Security Handbook (09).
- **Drainer-as-a-service (DaaS)**: a commercial criminal toolkit that automates sweeping a victim's approved balances after one malicious signature. Underlies WA04.
- **EIP-712**: an Ethereum Improvement Proposal standardizing hashed, typed structured data signing, so a wallet can render a request in human-readable fields instead of opaque hex. See the EIP-712 specification.
- **Multisig (multi-signature wallet)**: a wallet requiring several independent keys to sign before execution; Safe{Wallet} (formerly Gnosis Safe) is the most widely deployed implementation. See Chapter 5.1.
- **Pig butchering**: a long-con romance or social fraud that builds a fabricated relationship before steering the victim into a fraudulent crypto-investment platform, a pattern the FBI and Treasury's FinCEN both track and which frequently traces to forced-labor scam compounds in Southeast Asia. See background and Chapter 4.1.3.
- **Wrench attack**: a physical-coercion attack against a crypto holder, named for the xkcd "security" comic observing that a five-dollar wrench defeats cryptography. Tracked incidents

rose sharply through 2024 and 2025, including the January 2025 kidnapping of a Ledger co-founder. See the physical-attack tracker and Chapter 4.1.4.

16.3 dApp, Infrastructure, and Supply Chain Terms

These terms describe the delivery and networking layers a Web3 application depends on, covered in Chapters 6, 8, and 9.

- **Dependency confusion** and **typosquatting**: package-manager and domain attacks that trick a build or a user into resolving to an attacker-published or attacker-registered lookalike. Covered in depth in the CDN and Front-End Supply Chain Security Handbook (01); see Chapter 6.1.1.
- **DNS hijacking**: taking control of a domain's resolution, through registrar takeover, cache poisoning, or route hijacking, silently redirecting users to attacker infrastructure. Central to WA13. See the DNS and Hosting Security Handbook (02).
- **RPC (remote procedure call) node**: the server a wallet or dApp queries to read chain state and broadcast transactions; a spoofed endpoint can return falsified balances or simulation results. See Chapter 8.1.
- **Rug pull** and **token impersonation**: a token deployer draining value through an undisclosed privileged function, or a contract cloning a legitimate project's name and metadata to deceive holders and aggregators. Central to WA10. See Chapter 6.1.3.
- **Sequencer**: the component of a rollup or Layer 2 network that orders and batches transactions before settlement to the base chain; its compromise or centralization is a distinct node-layer risk. See Chapter 8.2.

16.4 Contract-Layer and Threat-Actor Terms

These terms describe the on-chain logic risks Chapter 7 maps onto SCWE and SC Top 10, and the organized adversaries behind Chapters 9 and 15.

- **Flash loan**: an uncollateralized loan borrowed and repaid within a single transaction, used to fund temporary price or governance-weight manipulation. Maps to SC04:2026.
- **Insider threat**: harm from a person with legitimate, authorized access, through deliberate collusive abuse or access that outlived its purpose. Central to WA12. See the Employee Lifecycle Security Handbook (03).
- **Lazarus Group**: a North Korean state-sponsored actor publicly attributed by the FBI to the February 2025 Bybit theft (roughly $1.4 to $1.5 billion in ether, the largest crypto theft on record). See background and Chapter 9.1.
- **Oracle manipulation** and **reentrancy**: skewing a thin-liquidity price feed within one transaction, or calling out to an external address before a state update finishes so the callee re-enters against stale state. Map to SC03:2026 and SC08:2026 respectively. See Chapter 7.2.

- **TraderTraitor**: the activity-cluster name US authorities use for DPRK-linked operations combining trojanized crypto applications with fake-recruiter social engineering, tracked jointly by CISA, the FBI, and Treasury. See the CISA advisory and Chapter 9.1.4.
- **DPRK IT worker fraud**: North Korean operatives obtaining remote developer roles under fabricated identities, often at crypto firms, to generate sanctioned revenue and, in a subset of cases, plant backdoors. Underlies WA15. See Chapter 9.1.4.

17. Appendix B: TTP Quick Reference Table

Part II builds one technique catalog per layer, each closing with a "Technique Catalog and TTP IDs" section (Chapters 4.4, 5.5, 6.5, 7.4, 8.3, and 9.5). This appendix pulls every entry into a single table so a reader threat-modeling a system, or triaging a live alert, does not have to page through six chapters to find the row that matches what they are looking at. TTP IDs follow a fixed pattern: a three-letter layer prefix (USR, WAL, APP, CON, NOD, INF) and a sequential two-digit number, matching the layer each entry's parent chapter covers. The **Tactic** column uses ATT&CK-style objective names (Chapter 1.2) so the table composes cleanly with any ATT&CK or AADAPT navigator you already run.

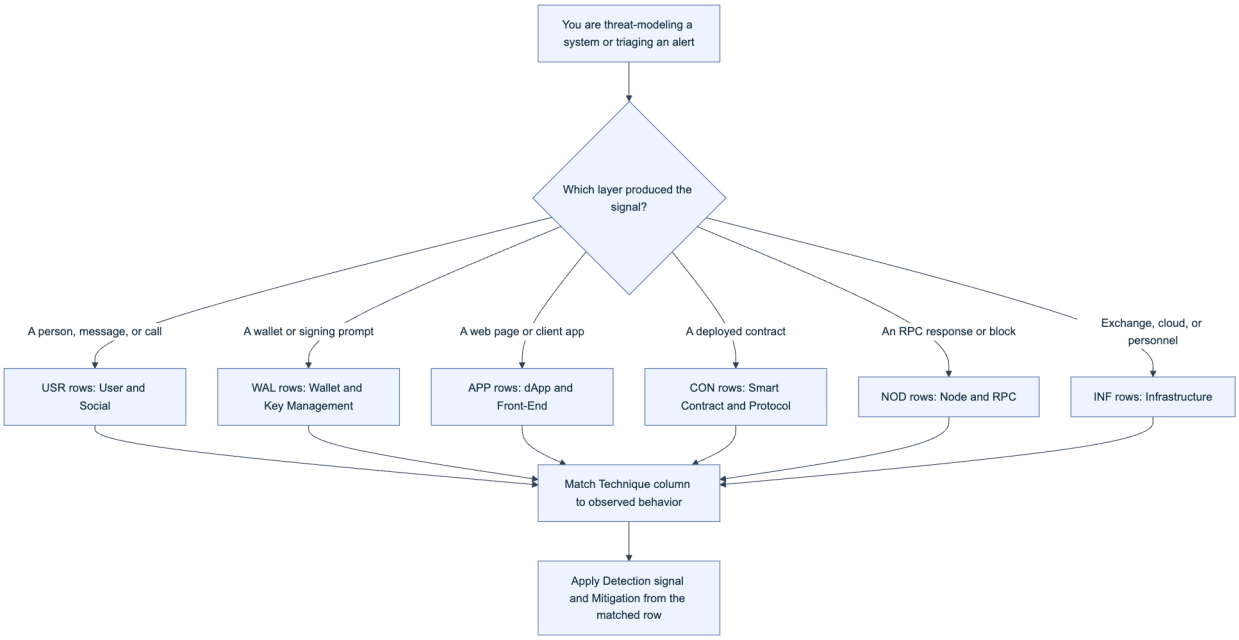

Figure 1. Layer-first lookup path through the TTP quick reference table. Most real incidents touch two or more layers; start from where the signal first surfaced, then read the Chapter cross-reference for the full write-up.

TTP ID	Layer	Tactic	Technique (summary)	WA / SC Ref	Detection Signal	Primary Mitigation	Chapter
USR-01	User and Social	Initial Access	Fake recruiter runs a video-call or "test task" that drops a loader disguised as meeting or build software	WA05	Unsigned installer downloaded right after a call; outbound to non-corporate infra	Sandboxed interviews; verify recruiters via a second channel	4.1.1
USR-02	User and Social	Initial Access	Lookalike-domain or compromised support-channel phishing harvests credentials or seed phrases	WA08	Domain-similarity monitoring; help-desk impersonation reports	Phishing-resistant MFA (FIDO2/WebAuthn); never request a seed phrase	4.1.2

TTP ID	Layer	Tactic	Technique (summary)	WA / SC Ref	Detection Signal	Primary Mitigation	Chapter
USR-03	User and Social	Initial Access, Impact	Long-con relationship building steers a victim into a fraudulent investment platform (pig butchering)	WA09	Deposit-velocity anomalies from dating/social-app-referred accounts	User education; exchange-side deposit and off-ramp coordination	4.1.3
USR-04	User and Social	Impact	Physical targeting and coercion of a known or presumed holder (wrench attack)	WA11	OSINT exposure monitoring; physical-security incident reports	Operational security discipline; duress wallets; time-locked withdrawals	4.1.4
WAL-01	Wallet and Key Mgmt	Execution, Impact	Malicious calldata substituted into a multisig UI so signers approve a different transaction than they see	WA01	Calldata decoding mismatches across independent signer devices	Hardware-wallet clear-signing; out-of-band hash confirmation	5.1.1
WAL-02	Wallet and Key Mgmt	Credential Access	Keyloggers, clipboard hijackers, or weak entropy expose a raw private key	WA03	Endpoint anomaly detection; clipboard-malware signatures	Hardware wallets; key sharding or MPC custody; entropy audits	5.1.2
WAL-03	Wallet and Key Mgmt	Impact	A drainer-as-a-service kit auto-sweeps every approved balance after one malicious signature	WA04	Simulation flags a mass `approve` or `setApprovalForAll` call	Wallet-side simulation; routine approval revocation	5.1.3
WAL-04	Wallet and Key Mgmt	Persistence	A malicious or compromised browser extension intercepts the signing flow	WA14	Extension permission audits; unexpected out-of-store updates	Extension allowlisting; hardware-wallet independent display	5.1.5
APP-01	dApp and Front-End	Initial Access	A typosquatted or maintainer-compromised OSS package injects wallet-draining code into the dependency tree	WA02	SBOM diffing on install; OSV/npm audit findings	Lockfile pinning; SRI; signed build provenance (Handbook 01)	6.1.1
APP-02	dApp and Front-End	Defense Evasion	A cloned or compromised front end swaps the recipient, spender, or amount mid-transaction (WA06's front-end face)	WA06	CSP violation reports; DOM-integrity monitoring	Subresource Integrity; strict-dynamic CSP; wallet-side simulation	6.1.2
APP-03	dApp and Front-End	Impact	A token contract with a hidden mint, pause, or blacklist function drains value post-launch	WA10	Bytecode diffing against verified source; honeypot scanners	Contract-verification requirements; liquidity-lock proof	6.1.3
APP-04	dApp and Front-End	Initial Access	Registrar takeover or route hijack redirects a dApp's domain to attacker infrastructure	WA13	DNSSEC validation failures; certificate-transparency alerts	Registry lock; DNSSEC; CAA records (Handbook 02)	6.1.4
CON-01	Smart Contract	Execution	An external call fires before the caller's own state update completes (reentrancy)	SC08:2026	Static-analyzer findings; invariant-fuzzing violations	Checks-effects-interactions ordering; reentrancy guards	7.2
CON-02	Smart Contract	Impact	A flash-loan-funded trade moves a thin-liquidity price feeding an unprotected oracle	SC03/04:2026	TWAP deviation alerts; oracle staleness checks	Time-weighted or multi-source oracles; circuit breakers	7.2, 7.3
CON-03	Smart Contract	Privilege Escalation	A missing or misconfigured access modifier leaves a privileged function callable by anyone	SC01:2026	Access-control matrix review; role-coverage tests	Explicit RBAC; least privilege; timelocked admin actions	7.2
NOD-01	Node and RPC	Collection, Impact	A spoofed RPC endpoint returns falsified balance or simulation data	infra	Multi-provider response cross-checking	Independent RPC providers; light-client verification	8.1
NOD-02	Node and RPC	Impact	A compromised validator or sequencer key enables censorship or double-signing	infra	Slashing-condition monitoring; consensus-divergence alerts	HSM-backed keys; redundant client software	8.2
INF-01	Infrastructure	Initial Access	Compromise of an exchange's back-office or cloud infrastructure enables unauthorized withdrawal	WA07	Privileged-access anomaly detection; withdrawal-pattern alerts	Cold-storage segregation; multi-party withdrawal approval	9.1.1
INF-02	Infrastructure	Collection, Exfiltration	A privileged insider abuses legitimate access to exfiltrate keys or approve fraud	WA12	User/entity behavior analytics; dual-control audit gaps	Least privilege; separation of duties; strict offboarding (Handbook 03)	9.1.2
INF-03	Infrastructure	Initial Access, Persistence	A state-linked operative obtains a developer role under a false identity, or backdoors a dependency	WA15	Identity-verification gaps; anomalous contributor/payroll patterns	Enhanced identity checks (Handbook 05); sanctions screening	9.1.4

Row APP-04 and the DNS-hijacking angle of infrastructure overlap deliberately: WA13 spans both the domain a dApp resolves to (dApp layer) and the registrar or routing infrastructure behind it (infrastructure layer). Treat a WA13 finding as two rows to check, not one.

18. Appendix C: OWASP Web3 Attack Vectors Top 15 (WA01–WA15) Summary Table

18.1 Full Vector List and Primary Targets

Use this table as the fast lookup for "which chapter covers WA-whatever." Every row names the primary layer this handbook assigns the vector to (a vector can touch more than one layer; the primary assignment is where its technique catalog lives) and points to the chapter section that carries the full treatment.

WA ID	Name	Primary Layer	Handbook Chapter	Representative Incident
WA01	Multisig Hijacking	Wallet and Key Management	5.1.1	Bybit, Feb 2025 (~$1.4-1.5B, Safe{Wallet} signer UI compromise)
WA02	Supply Chain Attacks (npm, PyPI, OSS)	dApp and Front-End	6.1.1	Ledger Connect Kit, Dec 2023
WA03	Private Key Compromise	Wallet and Key Management	5.1.2	Malware and clipboard-hijacker campaigns
WA04	Drainer Malware and Drainer-as-a-Service (DaaS)	Wallet and Key Management	5.1.3	Commercial drainer kits sold on underground forums
WA05	Fake Interview and Video Call Social Engineering	User and Social	4.1.1	DPRK-linked developer targeting
WA06	UI/UX Spoofing and Approval Phishing	Wallet / dApp	5.1.4, 6.1.2	Cloned front ends harvesting unlimited approvals
WA07	Centralised Exchange and Web2/2.5 Infrastructure Breaches	Infrastructure	9.1.1	Exchange back-office compromises
WA08	Phishing and General Social Engineering	User and Social	4.1.2	Ongoing, cross-platform
WA09	Romance, Investment, Impersonation, Recovery, and Pig Butchering Scams	User and Social	4.1.3	Southeast Asia scam-compound operations
WA10	Rug Pulls, Fake Airdrops, and Token Impersonation	dApp and Front-End	6.1.3	Recurring token-launch fraud
WA11	Wrench Attacks and Physical Coercion	User and Social	4.1.4	Ledger co-founder kidnapping, Jan 2025
WA12	Insider Threats and Collusive Abuse	Infrastructure	9.1.2	Employee-enabled fund diversion

WA ID	Name	Primary Layer	Handbook Chapter	Representative Incident
WA13	DNS, Domain, and Routing Infrastructure Hijacking	dApp / Infrastructure	6.1.4, 9.1.3	Registrar and DNS takeovers
WA14	Wallet Software, Extension, and App Compromises	Wallet and Key Management	5.1.5	Compromised browser-extension builds
WA15	Nation-State Infiltration via Fake Hiring and Malicious OSS Contributions	Infrastructure	9.1.4	DPRK IT worker fraud, TraderTraitor

18.1.1 Source: OWASP SCS, Alternate Top 15: Web3 Attack Vectors (Beyond Smart Contracts)

The canonical list lives at scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15, published by the OWASP Smart Contract Security (SCS) project as the deliberate complement to the Smart Contract Top 10. Where the SC Top 10 ranks on-chain logic risk, WA01 through WA15 covers everything outside the contract: custody, social engineering, front-end delivery, infrastructure, and the human layer above it.

Two scope notes matter when you cite this list. The OWASP project states explicitly that the ordering is not a severity or frequency ranking; WA01 is not "worse" than WA15, and a data-driven ranking methodology is flagged as a planned future addition. Treat the list as a coverage checklist, not a priority queue, until that ships. The list is also versioned alongside the SC Top 10, so wording or grouping can shift between editions; this handbook's chapter numbers point to the vector IDs, the stable reference, rather than to any single edition's prose. When you build a control against a specific WA ID, record the publication date you worked from and re-check the source on your next review cycle (Chapter 15.1 covers this handbook's own update discipline).

---

19. Appendix D: Mapping to SCWE and SC Top 10

The Web3 Attack Vectors Top 15 and the SC Top 10 are deliberately non-overlapping: the Top 15 covers what happens around the contract, the SC Top 10 covers what happens inside it. Most WA vectors therefore have no SCWE or SC Top 10 mapping at all, because their root cause never touches deployed bytecode. A handful sit at the seam, where an off-chain vector (a spoofed UI, a compromised deployer key, an insider with a privileged role) is the delivery mechanism for an on-chain weakness. This appendix is the quick-reference for that seam, consolidating the mapping Chapter 7.1 develops in full prose.

Figure 2. Only vectors in the seam group carry an SC Top 10 or SCWE mapping. The nine purely off-chain vectors are fully addressed by this handbook's own detection and mitigation columns (Appendix B); mapping them to a contract-weakness catalog would misrepresent where the risk lives.

WA ID	On-Chain Touchpoint	SC Top 10:2026	Representative SCWE Domain
WA01	Multisig executes an attacker-substituted call once signers are deceived	SC01:2026 (Access Control)	SCSVS-AUTH: access-control and authorization weaknesses
WA02	Compromised dependency ships malicious logic into a contract's build or a front end's signing path	SC05:2026 (Lack of Input Validation), SC10:2026 (Proxy and Upgradeability)	SCSVS-CODE: code and dependency management weaknesses
WA04	A single deceptive approval becomes an unbounded, contract-enforced allowance	SC02:2026 (Business Logic)	SCSVS-AUTH: approval and allowance handling weaknesses
WA06	Approval or transfer calldata differs from what the user believed they authorized	SC02:2026 (Business Logic)	SCSVS-AUTH: signature and approval handling weaknesses
WA10	Deployer retains an undisclosed privileged function (mint, pause, blacklist)	SC01:2026 (Access Control), SC02:2026 (Business Logic)	SCSVS-GOV: business-logic and privileged-role weaknesses
WA12	An insider with a legitimate privileged role executes or authorizes a malicious on-chain action	SC01:2026 (Access Control)	SCSVS-AUTH: access-control and role-management weaknesses

Read the table as a starting point, not an exhaustive cross-reference; a given incident can implicate more than one SC Top 10 category (the Bybit theft is a WA01 finding whose blast radius touched both access control and business logic). In a finding write-up, cite the specific SCWE entry ID your reviewer identifies rather than the domain grouping shown here; the domain grouping is a navigation aid, the individual SCWE-XXX entry is the citable weakness.

---

20. Appendix E: References and Further Reading

Every claim in this handbook that names a standard, a date, a figure, or an incident is cited inline at the point it is made, the convention Part I establishes. Rather than duplicate that source list here, this appendix points to `references.md`, the deduplicated bibliography for the full handbook, grouped into Standards and Frameworks (the Web3 Attack Vectors Top 15, SC Top 10, SCWE, SCSVS, SCSTG, MITRE ATT&CK and AADAPT, STRIDE), Incidents and Case Studies (Bybit,

Ledger Connect Kit, the Ledger co-founder kidnapping, TraderTraitor and DPRK IT worker fraud reporting), Tools (transaction simulators, honeypot and bytecode-diffing scanners, SBOM tooling from Handbook 01), and Further Reading (vendor post-mortems, tracker projects, and companion material outside this handbook's scope).

If a link anywhere in this handbook goes stale, check `references.md` first for an updated URL or an archived copy before assuming the underlying claim no longer holds. When you extend this handbook, whether adding a new TTP row, a new WA-to-SCWE mapping, or a new incident, add the source to `references.md` in the same change so the bibliography never drifts out of sync with the body text.

Key controls for Part 5

- Treat Appendix A as the shared vocabulary: link to it from tickets and post-mortems instead of redefining a term inline each time.
- Start any threat-modeling session or alert triage from the layer-first lookup in Appendix B (Figure 1), then follow the Chapter cross-reference to the full technique write-up.
- Cite the Web3 Attack Vectors Top 15 by WA ID, not by rank; the list is explicitly unranked (Appendix C, 18.1.1), and treating the numbering as a severity order misrepresents the source.
- When a finding sits in the seam between an off-chain vector and on-chain logic, cite the specific SCWE entry your review identifies, not just the domain grouping in Appendix D.
- Add every new citation to `references.md` in the same change that introduces it (Appendix E).

Appendix C: References and Further Reading

This appendix consolidates every external source cited across the Handbook's five parts, grouped by category for quick lookup.

Standards and Frameworks

- MITRE ATT&CK
- MITRE ATT&CK Design and Philosophy / Data and Tools
- MITRE ATT&CK Version History
- MITRE ATT&CK Tactic TA0001: Initial Access
- MITRE ATT&CK Tactic TA0005: Defense Evasion
- MITRE ATT&CK Tactic TA0006: Credential Access
- MITRE ATT&CK Tactic TA0009: Collection
- MITRE ATT&CK Tactic TA0040: Impact
- MITRE ATT&CK Tactic TA0043: Reconnaissance
- MITRE ATT&CK Technique T1078: Valid Accounts
- MITRE ATT&CK Technique T1195: Supply Chain Compromise
- MITRE ATT&CK Technique T1195.002: Compromise Software Supply Chain
- MITRE ATT&CK Technique T1552.004: Private Keys
- MITRE ATT&CK Technique T1566: Phishing
- MITRE ATT&CK Technique T1584: Compromise Infrastructure
- MITRE ATT&CK Technique T1584.001: Compromise Infrastructure - Domains
- MITRE AADAPT project site
- MITRE AADAPT GitHub repository
- MITRE AADAPT fact sheet: Cyber Threat Framework for Digital Assets
- NIST glossary: Tactics, Techniques, and Procedures (SP 800-150)
- OWASP Smart Contract Security (SCS)
- OWASP Smart Contract Security Verification Standard (SCSVS)
- OWASP Smart Contract Weakness Enumeration (SCWE)
- OWASP Smart Contract Top 10
- OWASP Web3 Attack Vectors Top 15
- OWASP SCS Contributing Guide
- EIP-712: Typed Structured Data Hashing and Signing
- EIP-2612: Permit Extension for EIP-20 Signed Approvals
- Microsoft Learn: Threat Modeling Tool threat categories

Incidents and Case Studies

- X (Twitter) post-incident update blog
- Wikipedia: Bybit
- Wikipedia: Pig butchering scam
- BBC News
- BleepingComputer: Trust Wallet confirms extension hack led to $7 million crypto theft
- CISA Advisory AA22-108A
- Chainalysis: Bybit exchange hack, February 2025 crypto security and DPRK analysis
- Chainalysis: Crypto drainers report
- CoinDesk: Harvest Finance incident coverage
- Gemini Cryptopedia: The DAO Hack
- Group-IB: Inferno Drainer
- FBI IC3 Public Service Announcement PSA 250226
- Ledger: Letter from Chairman/CEO Pascal Gauthier regarding the Ledger Connect Kit exploit
- NCC Group: In-depth technical analysis of the Bybit hack
- OpenSSF blog: XZ Utils backdoor coverage
- Parity blog: 2017 multisig wallet incident
- Reuters: Coincheck hack coverage, January 2018
- Sygnia: Investigation into the Bybit hack
- Wired: Mt. Gox bitcoin exchange collapse, 2014

Tools and Documentation

- CycloneDX
- Sigstore: Cosign signing overview
- DeFiHackLabs GitHub repository
- Physical Bitcoin Attacks tracker
- Scam Sniffer
- SPDX
- ICANN: Registration Data Access Protocol (RDAP)
- Security Alliance (SEAL)
- SEAL 911
- Businesswire: Security Alliance (SEAL) launches free crypto-native ISAC

Further Reading

- FBI Internet Crime Complaint Center (IC3)

- Chainalysis blog: Crypto Crime research
- ESET WeLiveSecurity
- Palo Alto Networks Unit 42
- xkcd 538, “Security”